

ĐỒ ÁN MÔN HỌC

ỨNG DỤNG APACHE SPARK TRONG PHÂN TÍCH DỮ LIỆU CHỨNG KHOÁN VÀ DỰ BÁO XU HƯỚNG GIÁ CỔ PHIẾU

Ngành: **KHOA HỌC DỮ LIỆU**

Chuyên ngành: **KHOA HỌC DỮ LIỆU**

Giảng viên hướng dẫn : ThS. Nguyễn Quang Phúc

Sinh viên thực hiện :

2286400908 - Nguyễn Ngọc Quỳnh Anh

2286400043 - Lê Hoàng Gia Vĩ

2286400044 - Hoàng Nguyên Vũ

Lớp: 22DKHA1

TP. Hồ Chí Minh, 2026

ĐỒ ÁN MÔN HỌC

ỨNG DỤNG APACHE SPARK TRONG PHÂN TÍCH DỮ LIỆU CHỨNG KHOÁN VÀ DỰ BÁO XU HƯỚNG GIÁ CỔ PHIẾU

Ngành: **KHOA HỌC DỮ LIỆU**

Chuyên ngành: **KHOA HỌC DỮ LIỆU**

Giảng viên hướng dẫn : ThS. Nguyễn Quang Phúc

Sinh viên thực hiện :

2286400908 - Nguyễn Ngọc Quỳnh Anh

2286400043 - Lê Hoàng Gia Vĩ

2286400044 - Hoàng Nguyên Vũ

Lớp: 22DKHA1

TP. Hồ Chí Minh, 2026

NHẬN XÉT CỦA GIÁNG VIÊN HƯỚNG DẪN

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

TP.HCM, Ngày Tháng Năm 2026

Giáo viên hướng dẫn

(Ký tên, đóng dấu)

LỜI CAM ĐOAN

Nhóm chúng tôi gồm Nguyễn Ngọc Quỳnh Anh, Lê Hoàng Gia Vĩ, và Hoàng Nguyễn Vũ xin cam đoan rằng:

Tất cả thông tin và kết quả nghiên cứu trong bài báo cáo này đều trung thực với khách quan, được thu thập từ các nguồn đáng tin cậy, chính thống. Chúng tôi đã phân tích kỹ lưỡng các tài liệu và đảm bảo rằng mọi dữ liệu hoặc ý kiến trích dẫn đều được ghi rõ ràng nguồn gốc, tuân thủ đúng quy định về trích dẫn học thuật.

Chúng tôi cam kết không có hành vi sao chép hoặc sử dụng trái phép bất kỳ thông tin nào từ các nguồn khác. Bài báo cáo này là sản phẩm nghiên cứu độc lập của nhóm, chủ yếu dựa trên công trình được thực hiện trước đây và tuân thủ đầy đủ các quy trình của môn học. Chúng tôi sử dụng công cụ nghiên cứu một cách hợp lý và chính xác, đảm bảo tính khoa học và độc lập trong quá trình thực hiện.

Nhóm chúng tôi hy vọng rằng bài báo cáo sẽ cung cấp cái nhìn sâu sắc về chủ đề “Ứng dụng Apache Spark trong phân tích dữ liệu chứng khoán và dự báo xu hướng giá cổ phiếu” và đóng góp tích cực vào phát triển lĩnh vực này.

TP.HCM, Ngày..... Tháng Năm 2026

Sinh viên

Nguyễn Ngọc Quỳnh Anh

Lê Hoàng Gia Vĩ

Hoàng Nguyễn Vũ

MỤC LỤC

1	Tổng quan	9
1.1	Giới thiệu đề tài	9
1.2	Nhiệm vụ của đề tài	9
1.2.1	Tính cấp thiết của đề tài	9
1.2.2	Ý nghĩa khoa học và thực tiễn của đề tài	9
1.3	Mục tiêu của đề tài	10
1.3.1	Mục tiêu tổng quát	10
1.3.2	Mục tiêu cụ thể	10
1.4	Đối tượng và phạm vi nghiên cứu	10
1.4.1	Đối tượng nghiên cứu	10
1.4.2	Phạm vi nghiên cứu	11
1.5	Những đóng góp của đề tài	11
2	Cơ sở lý thuyết	12
2.1	Tổng quan về Big Data	12
2.1.1	Khái niệm Big Data	12
2.1.2	Tính chất đặc trưng của Big Data	12
2.1.3	Kiến trúc Big Data	13
2.1.4	Ứng dụng của Big Data trong các lĩnh vực	14
2.1.5	Các công nghệ xử lý dữ liệu lớn	15
2.2	Apache Spark	16
2.2.1	Giới thiệu về Apache Spark	16
2.2.2	Kiến trúc Apache Spark	17
2.2.3	So sánh Apache Spark với Hadoop MapReduce	19
2.2.4	Các thành phần quan trọng trong Spark	20
3	Phương pháp thực nghiệm	23
3.1	Tổng quan phương pháp	23
3.2	Mô tả bộ dữ liệu (Dataset)	24
3.2.1	Nguồn dữ liệu	24
3.2.2	Đặc trưng dữ liệu	24
3.2.3	Quy mô và phạm vi dữ liệu	24
3.3	Tiền xử lý dữ liệu (Data Preprocessing)	25
3.3.1	Phân tích dữ liệu ban đầu (EDA)	25
3.3.2	Làm sạch dữ liệu (Data Cleaning)	26
3.3.3	Xử lý giá trị ngoại lai (Outlier Handling)	26

3.3.4	Biến đổi dữ liệu (Data Transformation)	26
3.3.5	Xây dựng đặc trưng	27
3.4	Xây dựng và huấn luyện mô hình (Modeling)	27
3.4.1	Chuẩn bị dữ liệu cho mô hình	27
3.4.2	Lựa chọn mô hình	27
3.4.3	Huấn luyện mô hình	27
3.4.4	Đánh giá mô hình	28
4	Kết quả thực nghiệm	29
4.1	Kết quả phân tích dữ liệu	29
4.1.1	Thống kê mô tả dữ liệu	29
4.1.2	Phân tích dữ liệu thiếu	30
4.1.3	Phân tích phân bố dữ liệu và xu hướng theo thời gian	30
4.2	Kết quả tiền xử lý dữ liệu	33
4.2.1	Loại bỏ các đặc trưng không cần thiết	33
4.2.2	Xử lý giá trị thiếu (Handling Missing Values)	33
4.2.3	Cắt dữ liệu theo mốc thời gian	33
4.2.4	Xử lý giá trị ngoại lai	33
4.2.5	Tạo đặc trưng mới	34
4.3	Kết quả xây dựng và huấn luyện mô hình	34
4.3.1	Kết quả đánh giá theo từng Fold	34
4.3.2	Đánh giá tổng thể mô hình	35
4.3.3	Kết quả Backtesting chiến lược giao dịch	36
4.3.4	Confusion matrix	36
5	Kết luận	37

DANH SÁCH HÌNH VẼ

1	Kiến trúc năm lớp của Big Data [1]	13
2	Quy trình thực thi Spark	19
3	Tổng quan phương pháp nghiên cứu	23
4	Biểu đồ đường thể hiện xu hướng biến động Giá đóng cửa trung bình giai đoạn 1962 - 2024.	31
5	So sánh giá trị trung bình của Giá và Khối lượng trước và sau năm 2000.	31
6	Phân phối của biến Volume trước và sau khi áp dụng phép biến đổi Log-Scale.	32
7	Biểu đồ Boxplot nhận diện các giá trị ngoại lai (Outliers) của biến Giá đóng cửa.	32
8	Biểu đồ đường thể hiện xu hướng biến động Giá đóng cửa trung bình giai đoạn 2000 - 2024.	34

DANH SÁCH BẢNG

1	Các đặc trưng của Big Data	12
2	Các ứng dụng của Big Data	15
3	So sánh Apache Spark và Hadoop MapReduce	20
4	Mô tả các thuộc tính dữ liệu cổ phiếu	24
5	Các bước phân tích dữ liệu	25
6	Thống kê tổng quan dữ liệu	29
7	Cấu trúc schema dữ liệu	29
8	Thống kê mô tả các biến giá và khối lượng	30
9	Kết quả đánh giá mô hình theo từng giai đoạn (Walk-forward Validation)	35
10	Kết quả đánh giá tổng thể mô hình.	35
11	Confusion Matrix của mô hình Random Forest	36

CHƯƠNG 1: TỔNG QUAN

1.1 Giới thiệu đề tài

Trong bối cảnh sự phát triển mạnh mẽ của công nghệ thông tin, lượng dữ liệu được tạo ra ngày càng lớn với tốc độ nhanh và đa dạng về định dạng. Đặc biệt trong lĩnh vực tài chính – chứng khoán, dữ liệu không chỉ bao gồm giá cổ phiếu theo thời gian mà còn có các yếu tố liên quan như khối lượng giao dịch, tin tức, và các chỉ số kinh tế. Việc khai thác và phân tích hiệu quả nguồn dữ liệu này đóng vai trò quan trọng trong việc hỗ trợ ra quyết định đầu tư.

Tuy nhiên, các phương pháp xử lý dữ liệu truyền thống gặp nhiều hạn chế khi phải xử lý khối lượng dữ liệu lớn (Big Data). Do đó, các nền tảng xử lý dữ liệu phân tán như Apache Spark đã ra đời, cho phép xử lý dữ liệu nhanh chóng và hiệu quả hơn.

Xuất phát từ nhu cầu thực tiễn đó, đề tài “Xây dựng hệ thống phân tích và dự báo giá cổ phiếu sử dụng Apache Spark trên dữ liệu lớn” được thực hiện nhằm nghiên cứu và ứng dụng các công nghệ Big Data trong bài toán phân tích và dự báo giá cổ phiếu.

1.2 Nhiệm vụ của đề tài

1.2.1 Tính cấp thiết của đề tài

Trong những năm gần đây, thị trường chứng khoán phát triển mạnh mẽ cả về quy mô lẫn mức độ phức tạp, kéo theo sự gia tăng nhanh chóng của khối lượng dữ liệu. Dữ liệu chứng khoán không chỉ lớn về số lượng mà còn có tính chất biến động liên tục theo thời gian thực, đòi hỏi các phương pháp xử lý phải có khả năng đáp ứng về tốc độ và hiệu năng.

Các phương pháp xử lý dữ liệu truyền thống ngày càng bộc lộ những hạn chế khi phải xử lý với dữ liệu lớn, đặc biệt trong các bài toán yêu cầu phân tích nhanh và dự báo chính xác như dự báo giá cổ phiếu. Trong khi đó, các công nghệ Big Data, điển hình là Apache Spark, cho phép xử lý dữ liệu theo mô hình phân tán với hiệu suất cao, phù hợp với các bài toán dữ liệu lớn hiện nay.

Vì vậy, việc nghiên cứu và ứng dụng Apache Spark vào bài toán phân tích và dự báo giá cổ phiếu là cần thiết, không chỉ nhằm nâng cao hiệu quả xử lý dữ liệu mà còn góp phần cải thiện độ chính xác trong dự báo, đáp ứng nhu cầu thực tiễn của thị trường.

1.2.2 Ý nghĩa khoa học và thực tiễn của đề tài

Về mặt khoa học, đề tài góp phần hệ thống hóa các kiến thức nền tảng liên quan đến Big Data và Apache Spark, đồng thời làm rõ khả năng ứng dụng của các thành phần trong Spark như Spark SQL, MLlib, Streaming và GraphX trong việc xử lý và phân tích dữ liệu lớn. Bên cạnh đó, đề tài còn đề xuất một quy trình xử lý dữ liệu và xây dựng mô

hình dự báo dựa trên nền tảng dữ liệu lớn, góp phần bổ sung tài liệu tham khảo cho các nghiên cứu liên quan.

Về mặt thực tiễn, đề tài hướng đến việc xây dựng một hệ thống hỗ trợ phân tích và dự báo giá cổ phiếu dựa trên dữ liệu lịch sử, qua đó giúp nhà đầu tư có thêm cơ sở trong việc ra quyết định. Ngoài ra, kết quả của đề tài có thể được mở rộng và áp dụng trong nhiều lĩnh vực khác có sử dụng dữ liệu lớn như thương mại điện tử, tài chính, y tế hoặc marketing, góp phần nâng cao hiệu quả khai thác và sử dụng dữ liệu trong thực tế.

1.3 Mục tiêu của đề tài

1.3.1 Mục tiêu tổng quát

Mục tiêu tổng quát của đề tài là nghiên cứu và ứng dụng các công nghệ Big Data, đặc biệt là Apache Spark, nhằm xây dựng một hệ thống có khả năng xử lý, phân tích và dự báo giá cổ phiếu dựa trên dữ liệu lịch sử. Thông qua đó, đề tài hướng đến việc nâng cao hiệu quả khai thác dữ liệu lớn trong lĩnh vực chứng khoán và hỗ trợ quá trình ra quyết định đầu tư.

1.3.2 Mục tiêu cụ thể

Để đạt được mục tiêu tổng quát, đề tài tập trung vào các mục tiêu cụ thể sau:

- Tìm hiểu và hệ thống hóa các kiến thức về Big Data và kiến trúc Apache Spark.
- Phân tích các thành phần chính của Apache Spark như Spark SQL, MLlib, Streaming và GraphX.
- Thu thập và tiền xử lý dữ liệu chứng khoán từ các nguồn dữ liệu lịch sử.
- Xây dựng hệ thống xử lý dữ liệu lớn dựa trên nền tảng Apache Spark.
- Ứng dụng các mô hình học máy trong MLlib để xây dựng mô hình dự báo giá cổ phiếu.
- Đánh giá hiệu quả của mô hình dự báo thông qua các chỉ số phù hợp.
- Đề xuất hướng cải tiến và mở rộng hệ thống trong tương lai.

1.4 Đối tượng và phạm vi nghiên cứu

1.4.1 Đối tượng nghiên cứu

Đối tượng nghiên cứu của đề tài bao gồm dữ liệu chứng khoán và các công nghệ liên quan đến xử lý dữ liệu lớn. Cụ thể, đề tài tập trung vào dữ liệu giá cổ phiếu theo thời gian, bao gồm các thuộc tính như giá mở cửa, giá đóng cửa, giá cao nhất, giá thấp nhất và khối lượng giao dịch.

Bên cạnh đó, đề tài nghiên cứu các công nghệ xử lý dữ liệu lớn, đặc biệt là nền tảng Apache Spark cùng các thành phần quan trọng như Spark SQL, MLlib, Streaming và GraphX. Ngoài ra, các mô hình học máy phục vụ cho bài toán dự báo cũng là một phần quan trọng trong đối tượng nghiên cứu.

1.4.2 Phạm vi nghiên cứu

Về phạm vi, đề tài giới hạn trong việc sử dụng dữ liệu lịch sử của một hoặc một số mã cổ phiếu nhằm phục vụ cho bài toán phân tích và dự báo. Việc dự báo chủ yếu tập trung vào xu hướng giá cổ phiếu trong ngắn hạn dựa trên dữ liệu quá khứ.

Đề tài tập trung triển khai các thành phần cốt lõi của Apache Spark, đặc biệt là Spark SQL (DataFrame) và MLlib trong xử lý và xây dựng mô hình dự báo. Các thành phần như Streaming và GraphX có thể được xem xét áp dụng ở mức cơ bản hoặc minh họa.

Ngoài ra, đề tài chưa đi sâu vào việc phân tích các yếu tố vĩ mô như chính sách kinh tế, tin tức thị trường hay tâm lý nhà đầu tư, mà chủ yếu tập trung vào dữ liệu định lượng có sẵn. Kết quả của đề tài mang tính chất nghiên cứu và thử nghiệm, làm cơ sở cho các hướng phát triển và mở rộng trong tương lai.

1.5 Những đóng góp của đề tài

Đề tài hướng đến việc đóng góp cả về mặt lý thuyết lẫn ứng dụng thực tiễn trong lĩnh vực phân tích dữ liệu lớn và tài chính. Trước hết, về mặt lý thuyết, đề tài góp phần hệ thống hóa các kiến thức nền tảng liên quan đến Big Data và Apache Spark, đồng thời làm rõ vai trò và khả năng của các thành phần trong Spark như Spark SQL, MLlib, Streaming và GraphX trong việc xử lý và phân tích dữ liệu lớn.

Bên cạnh đó, đề tài đề xuất một quy trình tổng thể cho việc xử lý dữ liệu lớn trong bài toán phân tích chứng khoán, bao gồm các bước từ thu thập dữ liệu, tiền xử lý, lưu trữ, đến xây dựng và đánh giá mô hình dự báo. Quy trình này có thể được xem như một tài liệu tham khảo cho các nghiên cứu và ứng dụng tương tự.

Về mặt thực tiễn, đề tài xây dựng một hệ thống thử nghiệm ứng dụng Apache Spark trong việc phân tích và dự báo giá cổ phiếu dựa trên dữ liệu lịch sử. Hệ thống này không chỉ giúp minh họa khả năng xử lý dữ liệu lớn của Spark mà còn cung cấp một công cụ hỗ trợ cơ bản cho việc ra quyết định trong đầu tư.

Ngoài ra, kết quả của đề tài có thể được mở rộng và phát triển trong tương lai, chẳng hạn như tích hợp thêm dữ liệu thời gian thực, cải tiến mô hình học máy hoặc mở rộng sang các lĩnh vực khác có dữ liệu lớn. Điều này góp phần khẳng định tính ứng dụng và khả năng phát triển lâu dài của đề tài.

CHƯƠNG 2: CƠ SỞ LÝ THUYẾT

2.1 Tổng quan về Big Data

2.1.1 Khái niệm Big Data

Trong những năm gần đây, sự phát triển mạnh mẽ của công nghệ thông tin và Internet đã làm gia tăng đáng kể lượng dữ liệu được tạo ra trong mọi lĩnh vực của đời sống. Dữ liệu không chỉ đến từ các hệ thống truyền thống như cơ sở dữ liệu doanh nghiệp mà còn được sinh ra liên tục từ mạng xã hội, thiết bị di động, cảm biến IoT, hệ thống giao dịch và nhiều nguồn khác. Điều này dẫn đến sự xuất hiện của một khái niệm mới được gọi là Big Data.

Big Data được hiểu là tập hợp các dữ liệu có kích thước rất lớn, tốc độ sinh ra nhanh và có cấu trúc đa dạng, vượt quá khả năng xử lý của các hệ thống quản lý dữ liệu truyền thống. Theo ACCA [2], Big Data không chỉ đề cập đến khối lượng dữ liệu mà còn bao gồm khả năng thu thập, lưu trữ, quản lý và phân tích dữ liệu để tạo ra thông tin có giá trị. Nói cách khác, Big Data không đơn thuần là “dữ liệu lớn” mà là một hệ sinh thái bao gồm dữ liệu, công nghệ và phương pháp phân tích nhằm khai thác tri thức từ dữ liệu.

Một điểm quan trọng cần nhấn mạnh là Big Data không chỉ được xác định bởi kích thước dữ liệu. Trong thực tế, một tập dữ liệu dù không quá lớn nhưng nếu có độ phức tạp cao, đa dạng về định dạng hoặc yêu cầu xử lý nhanh cũng có thể được xem là Big Data. Theo TechTarget [3], Big Data thường bao gồm dữ liệu có cấu trúc, bán cấu trúc và phi cấu trúc, đòi hỏi các công nghệ xử lý mới như hệ thống phân tán và xử lý song song.

2.1.2 Tính chất đặc trưng của Big Data

Big Data thường được đặc trưng bởi mô hình 5V [3] [4], thể hiện các đặc điểm quan trọng của dữ liệu lớn. Những đặc điểm này giúp phân biệt Big Data với các hệ thống dữ liệu truyền thống, đồng thời định hướng cho việc lựa chọn công nghệ và phương pháp xử lý phù hợp.

Bảng dưới đây tóm tắt các đặc trưng chính của Big Data:

Bảng 1: Các đặc trưng của Big Data

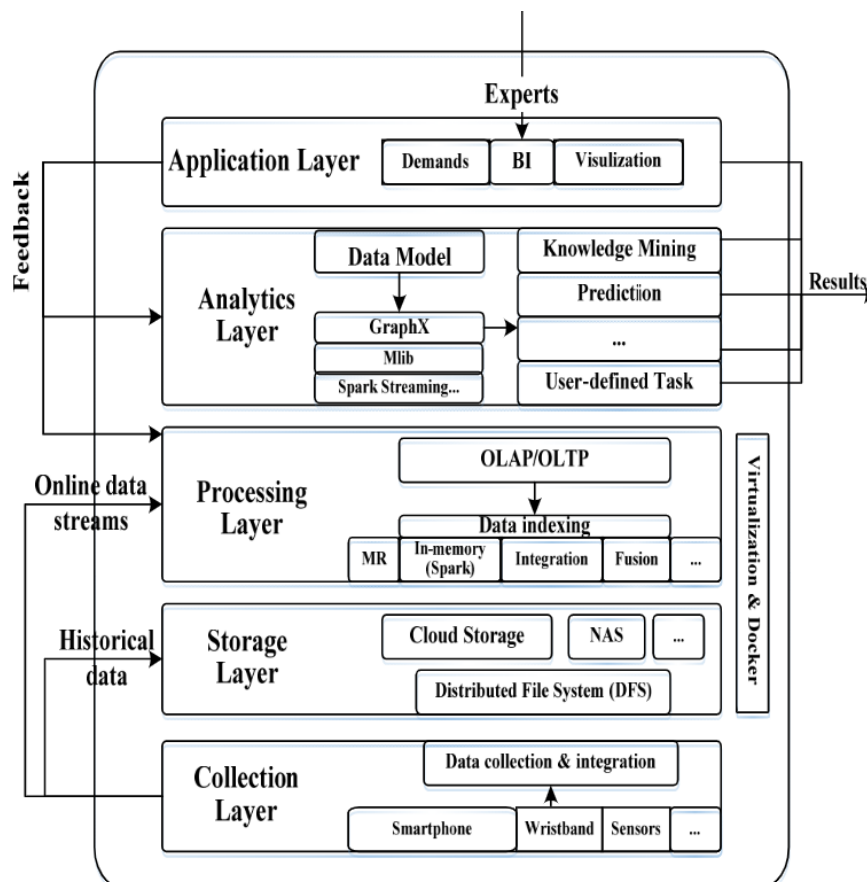
Đặc trưng	Mô tả
Volume (Khối lượng)	Dữ liệu có kích thước rất lớn, từ terabyte đến petabyte hoặc hơn, vượt quá khả năng lưu trữ của hệ thống truyền thống.
Velocity (Tốc độ)	Dữ liệu được tạo ra và xử lý với tốc độ cao, nhiều trường hợp yêu cầu xử lý gần thời gian thực.

Đặc trưng	Mô tả
Variety (Đa dạng)	Dữ liệu tồn tại dưới nhiều dạng như có cấu trúc, bán cấu trúc và phi cấu trúc (văn bản, hình ảnh, video, ...).
Veracity (Độ tin cậy)	Dữ liệu có thể chứa nhiễu, thiếu hoặc không chính xác, cần được làm sạch trước khi phân tích.
Value (Giá trị)	Khả năng khai thác dữ liệu để tạo ra thông tin hữu ích, hỗ trợ ra quyết định.

Nhìn chung, năm đặc trưng này không tồn tại độc lập mà có mối liên hệ chặt chẽ với nhau. Chẳng hạn, khi khối lượng dữ liệu tăng lên (Volume), yêu cầu về tốc độ xử lý (Velocity) và công nghệ lưu trữ cũng trở nên quan trọng hơn. Do đó, việc hiểu rõ các đặc trưng của Big Data là cơ sở để xây dựng các hệ thống xử lý dữ liệu hiệu quả.

2.1.3 Kiến trúc Big Data

Để xử lý hiệu quả dữ liệu lớn với các đặc trưng về khối lượng, tốc độ và tính đa dạng, các hệ thống Big Data thường được thiết kế theo kiến trúc phân tán và phân tầng. Kiến trúc này cho phép dữ liệu được thu thập, lưu trữ, xử lý và phân tích một cách linh hoạt trên nhiều hệ thống khác nhau, từ đó nâng cao hiệu năng và khả năng mở rộng [1].



Hình 1: Kiến trúc năm lớp của Big Data [1]

Một hệ thống Big Data điển hình thường được tổ chức theo nhiều tầng chức năng liên kết chặt chẽ với nhau. Quá trình bắt đầu từ việc thu thập dữ liệu từ nhiều nguồn khác nhau như hệ thống giao dịch, mạng xã hội, thiết bị cảm biến hoặc các tệp nhật ký. Dữ liệu sau khi thu thập sẽ được đưa vào hệ thống lưu trữ phân tán, nơi cho phép lưu trữ khối lượng dữ liệu lớn một cách linh hoạt và an toàn, thường sử dụng các công nghệ như hệ thống tệp phân tán hoặc cơ sở dữ liệu NoSQL.

Tiếp theo, dữ liệu được xử lý thông qua các nền tảng tính toán phân tán. Tùy theo yêu cầu, dữ liệu có thể được xử lý theo từng lô hoặc theo thời gian thực nhằm đáp ứng các bài toán khác nhau. Sau khi xử lý, dữ liệu sẽ được đưa vào các mô hình phân tích nhằm trích xuất thông tin, phát hiện xu hướng hoặc hỗ trợ dự báo. Cuối cùng, kết quả được trình bày dưới dạng trực quan như biểu đồ hoặc báo cáo để hỗ trợ người dùng trong quá trình ra quyết định.

Theo Chen và cộng sự [5], kiến trúc phân tầng và phân tán của Big Data không chỉ giúp xử lý hiệu quả dữ liệu lớn mà còn đảm bảo tính linh hoạt và khả năng mở rộng của hệ thống. Nhờ đó, các tổ chức có thể dễ dàng thích ứng với sự gia tăng liên tục của dữ liệu trong thực tế.

2.1.4 Ứng dụng của Big Data trong các lĩnh vực

Big Data đã trở thành một công nghệ cốt lõi với phạm vi ứng dụng rộng khắp trong nhiều lĩnh vực khác nhau của đời sống và kinh tế. Nhờ khả năng thu thập, lưu trữ và phân tích dữ liệu khổng lồ, Big Data giúp các tổ chức đưa ra quyết định chính xác, tối ưu hóa hoạt động và dự đoán xu hướng trong tương lai.

Trong lĩnh vực y tế, Big Data đóng vai trò quan trọng trong việc cải thiện chất lượng chăm sóc sức khỏe và hỗ trợ ra quyết định lâm sàng. Dữ liệu được thu thập từ hồ sơ bệnh án, thiết bị y tế, cảm biến và nghiên cứu sinh học được phân tích để dự đoán bệnh, chẩn đoán sớm và cá nhân hóa điều trị [6], [7]. Ngoài ra, Big Data còn hỗ trợ giám sát dịch bệnh, nghiên cứu dịch tễ học và tối ưu hóa chi phí y tế, góp phần nâng cao hiệu quả của hệ thống chăm sóc sức khỏe [8].

Trong lĩnh vực tài chính, Big Data được sử dụng để phân tích hành vi khách hàng, phát hiện gian lận và quản lý rủi ro. Các tổ chức tài chính khai thác dữ liệu giao dịch để dự đoán xu hướng thị trường và đưa ra quyết định đầu tư chính xác hơn [9]. Bên cạnh đó, Big Data còn giúp cải thiện dịch vụ khách hàng thông qua cá nhân hóa sản phẩm tài chính và tối ưu hóa quy trình vận hành.

Big Data đóng vai trò quan trọng trong việc phân tích hành vi người tiêu dùng và xây dựng chiến lược marketing hiệu quả. Các doanh nghiệp sử dụng dữ liệu từ mạng xã hội, thương mại điện tử và các nền tảng số để hiểu nhu cầu khách hàng, từ đó tối ưu hóa chiến dịch quảng cáo và tăng doanh thu [10]. Ngoài ra, Big Data còn hỗ trợ quản lý chuỗi cung ứng, dự báo nhu cầu và tối ưu hóa hoạt động logistics.

Trong nông nghiệp, Big Data được ứng dụng trong mô hình nông nghiệp thông minh (precision agriculture). Dữ liệu từ cảm biến, vệ tinh và thiết bị IoT giúp nông dân theo dõi điều kiện đất, thời tiết và cây trồng để tối ưu hóa sản xuất [11]. Việc áp dụng Big Data giúp nâng cao năng suất, giảm chi phí và cải thiện chất lượng sản phẩm nông nghiệp.

Big Data là nền tảng quan trọng trong việc xây dựng thành phố thông minh. Dữ liệu từ các hệ thống cảm biến và thiết bị IoT được sử dụng để quản lý giao thông, tối ưu hóa năng lượng và nâng cao chất lượng cuộc sống đô thị [12]. Trong giao thông, Big Data giúp dự đoán tình trạng ùn tắc, tối ưu hóa lộ trình và nâng cao hiệu quả vận tải.

Big Data đóng vai trò quan trọng trong việc phát hiện và ngăn chặn các mối đe dọa an ninh mạng. Thông qua phân tích dữ liệu thời gian thực, các hệ thống có thể nhận diện hành vi bất thường và phản ứng nhanh với các cuộc tấn công mạng [12].

Bảng 2: Các ứng dụng của Big Data

Lĩnh vực	Ứng dụng	Lợi ích
Y tế	Phân tích hồ sơ bệnh án, dự đoán bệnh, cá nhân hóa điều trị	Nâng cao chất lượng chăm sóc, giảm chi phí y tế
Tài chính – Ngân hàng	Phát hiện gian lận, phân tích rủi ro, dự đoán thị trường	Tăng độ chính xác trong quyết định, giảm rủi ro
Kinh doanh	Phân tích hành vi khách hàng, tối ưu marketing	Tăng doanh thu, cải thiện trải nghiệm khách hàng
Nông nghiệp	Giám sát cây trồng, phân tích môi trường	Tăng năng suất, giảm chi phí sản xuất
Giao thông	Dự đoán ùn tắc, tối ưu lộ trình	Giảm tắc nghẽn, nâng cao hiệu quả vận tải
Giáo dục	Phân tích hành vi học tập, cá nhân hóa nội dung	Nâng cao hiệu quả đào tạo
An ninh mạng	Phát hiện bất thường, phân tích tấn công	Tăng cường bảo mật hệ thống

2.1.5 Các công nghệ xử lý dữ liệu lớn

Để xử lý dữ liệu lớn, nhiều công nghệ và nền tảng đã được phát triển, cho phép lưu trữ và xử lý dữ liệu theo mô hình phân tán, hỗ trợ cả xử lý theo lô và thời gian thực. Các công nghệ tiêu biểu bao gồm:

- Apache Hadoop: Là nền tảng xử lý dữ liệu phân tán phổ biến, sử dụng mô hình lập trình MapReduce kết hợp với hệ thống lưu trữ HDFS. Hadoop có khả năng mở rộng cao và phù hợp với các bài toán xử lý batch. Tuy nhiên, do phụ thuộc vào việc đọc/ghi dữ liệu từ đĩa, hiệu năng của Hadoop bị hạn chế trong các bài toán yêu cầu

xử lý nhanh [13].

- Apache Spark: Là framework xử lý dữ liệu lớn hiện đại với hiệu năng cao nhờ cơ chế in-memory processing. So với Hadoop, Spark có thể xử lý nhanh hơn đáng kể do hạn chế truy cập đĩa. Ngoài ra, Spark hỗ trợ nhiều mô hình xử lý như batch, streaming, machine learning và xử lý dữ liệu có cấu trúc thông qua Spark SQL. Cấu trúc dữ liệu RDD (Resilient Distributed Dataset) giúp đảm bảo khả năng chịu lỗi và xử lý linh hoạt. Nhờ những ưu điểm này, Spark trở thành công nghệ cốt lõi trong nhiều hệ thống Big Data hiện đại [14].
- NoSQL Databases: Các hệ quản trị cơ sở dữ liệu như MongoDB và Apache Cassandra được thiết kế để lưu trữ dữ liệu phi cấu trúc hoặc bán cấu trúc. So với cơ sở dữ liệu quan hệ truyền thống, NoSQL cung cấp khả năng mở rộng linh hoạt và hiệu quả hơn trong môi trường dữ liệu lớn [15].
- Apache Kafka: Apache Kafka là nền tảng xử lý dữ liệu dòng (streaming) với khả năng truyền tải và xử lý dữ liệu theo thời gian thực. Kafka thường được sử dụng trong các hệ thống yêu cầu độ trễ thấp như phân tích log, hệ thống giám sát và pipeline dữ liệu [16].
- Data Warehouse và Data Lake: Đây là hai mô hình lưu trữ quan trọng trong Big Data. Data Warehouse tập trung vào dữ liệu có cấu trúc, phục vụ phân tích và báo cáo, trong khi Data Lake cho phép lưu trữ dữ liệu ở nhiều định dạng khác nhau, hỗ trợ tốt cho các bài toán phân tích nâng cao và machine learning [17].

Nhìn chung, các công nghệ xử lý dữ liệu lớn đóng vai trò quan trọng trong việc xây dựng hệ sinh thái Big Data. Trong đó, Apache Spark nổi bật nhờ hiệu năng cao, khả năng xử lý linh hoạt và hỗ trợ nhiều mô hình tính toán, do đó được lựa chọn làm nền tảng chính trong nhiều nghiên cứu và ứng dụng thực tế.

2.2 Apache Spark

2.2.1 Giới thiệu về Apache Spark

Trong bối cảnh nhu cầu xử lý dữ liệu lớn ngày càng tăng, các hệ thống truyền thống như Hadoop MapReduce đã bộc lộ nhiều hạn chế về hiệu năng, đặc biệt là trong các bài toán yêu cầu xử lý nhanh hoặc thời gian thực. Để khắc phục những hạn chế này, Apache Spark đã được phát triển như một nền tảng xử lý dữ liệu phân tán với hiệu năng cao.

Apache Spark là một hệ thống xử lý dữ liệu mã nguồn mở, cho phép thực hiện các phép tính song song trên cụm máy tính. Điểm nổi bật của Spark là khả năng xử lý dữ liệu trong bộ nhớ (in-memory processing), giúp tăng tốc độ xử lý đáng kể so với các hệ thống dựa trên đĩa như Hadoop MapReduce [18]. Nhờ đó, Spark đặc biệt phù hợp với các bài toán yêu cầu lặp lại nhiều lần như học máy hoặc phân tích dữ liệu tương tác.

Ngoài ra, Spark còn hỗ trợ nhiều loại dữ liệu và nhiều mô hình xử lý khác nhau, từ xử lý theo lô (batch processing) đến xử lý dòng dữ liệu thời gian thực (stream processing), giúp nó trở thành một công cụ linh hoạt trong hệ sinh thái Big Data [19].

2.2.2 Kiến trúc Apache Spark

Apache Spark được thiết kế theo mô hình tính toán phân tán dựa trên kiến trúc driver-worker, trong đó một tiến trình trung tâm điều phối việc thực thi và nhiều tiến trình thực thi phân tán xử lý dữ liệu song song. Kiến trúc này cho phép Spark đạt hiệu năng cao nhờ tận dụng bộ nhớ và khả năng xử lý song song trên nhiều node trong cluster.

Driver Program

Driver là thành phần trung tâm của một ứng dụng Spark, chịu trách nhiệm điều phối toàn bộ quá trình thực thi. Driver chạy hàm main() của chương trình và đóng vai trò như bộ não của hệ thống.

Cụ thể, Driver thực hiện các chức năng sau:

- Khởi tạo SparkSession/SparkContext để giao tiếp với cluster.
- Chuyển đổi code người dùng thành một kế hoạch thực thi (execution plan).
- Xây dựng Directed Acyclic Graph (DAG) biểu diễn luồng xử lý dữ liệu.
- Phân chia công việc thành các stage và task.
- Gửi task đến các Executors và theo dõi trạng thái thực thi.
- Thu thập kết quả trả về từ các Executors.

Driver đóng vai trò quan trọng trong việc lập lịch và tối ưu hóa quá trình xử lý dữ liệu. Nếu Driver gặp lỗi, toàn bộ ứng dụng Spark sẽ dừng lại [20].

Executors

Executors là các tiến trình chạy trên các worker nodes trong cluster, chịu trách nhiệm thực thi các task được Driver phân phối. Mỗi ứng dụng Spark sẽ có một tập hợp Executors riêng biệt và không chia sẻ với các ứng dụng khác.

Các đặc điểm và chức năng chính của Executors bao gồm:

- Thực thi task song song: Mỗi Executor có thể chạy nhiều task đồng thời thông qua nhiều thread, tùy thuộc vào số lượng CPU cores được cấp phát.
- Xử lý dữ liệu theo partition: Dữ liệu đầu vào được chia thành nhiều partition, mỗi task xử lý một partition. Điều này cho phép Spark đạt được mức độ song song cao.
- Lưu trữ dữ liệu trung gian: Executors có khả năng lưu trữ dữ liệu trung gian trong bộ nhớ (RAM) thông qua cơ chế caching hoặc persistence, giúp giảm chi phí truy cập đĩa và tăng tốc độ xử lý.

- **Giao tiếp với Driver:** Executors gửi kết quả, trạng thái và thông tin lỗi về Driver. Driver sử dụng thông tin này để điều phối và tối ưu quá trình thực thi.
- **Quản lý tài nguyên:** Mỗi Executor được cấp phát một lượng tài nguyên cố định (CPU, memory). Việc phân bổ tài nguyên này ảnh hưởng trực tiếp đến hiệu năng của ứng dụng Spark.

Ngoài ra, Spark còn tối ưu hóa việc phân phối task dựa trên data locality, tức là ưu tiên thực thi task trên node chứa dữ liệu, nhằm giảm chi phí truyền dữ liệu qua mạng [20].

DAG Scheduler

DAG Scheduler là thành phần quan trọng trong cơ chế lập lịch của Spark, chịu trách nhiệm chuyển đổi kế hoạch xử lý dữ liệu thành các đơn vị thực thi cụ thể [20].

- **Xây dựng DAG:** Khi người dùng định nghĩa các transformation (như *map*, *filter*, *join*), Spark không thực thi ngay mà chỉ ghi nhận các thao tác này để xây dựng một DAG logic. DAG này biểu diễn toàn bộ pipeline xử lý dữ liệu.
- **Phân chia Stage:** DAG Scheduler phân tích DAG và chia thành các stage dựa trên loại dependency giữa các phép toán:
 - **Narrow dependency:** Mỗi partition của dữ liệu đầu ra phụ thuộc vào một partition của dữ liệu đầu vào (ví dụ: *map*, *filter*). Các phép toán này không yêu cầu shuffle.
 - **Wide dependency:** Một partition đầu ra phụ thuộc vào nhiều partition đầu vào (ví dụ: *groupByKey*, *reduceByKey*). Các phép toán này yêu cầu shuffle, tức là truyền dữ liệu qua mạng.

DAG Scheduler sẽ tách DAG tại các điểm xảy ra wide dependency để tạo thành các stage độc lập.

- **Tạo Task:** Mỗi stage được chia thành nhiều task, với mỗi task xử lý một partition dữ liệu. Các task trong cùng một stage có thể chạy song song trên các Executors.
- **Lập lịch thực thi:** Spark sử dụng mô hình lập lịch hai tầng:
 - DAG Scheduler (high-level): Quản lý stage và dependency giữa các stage
 - Task Scheduler (low-level): Phân phối task cụ thể đến các Executors

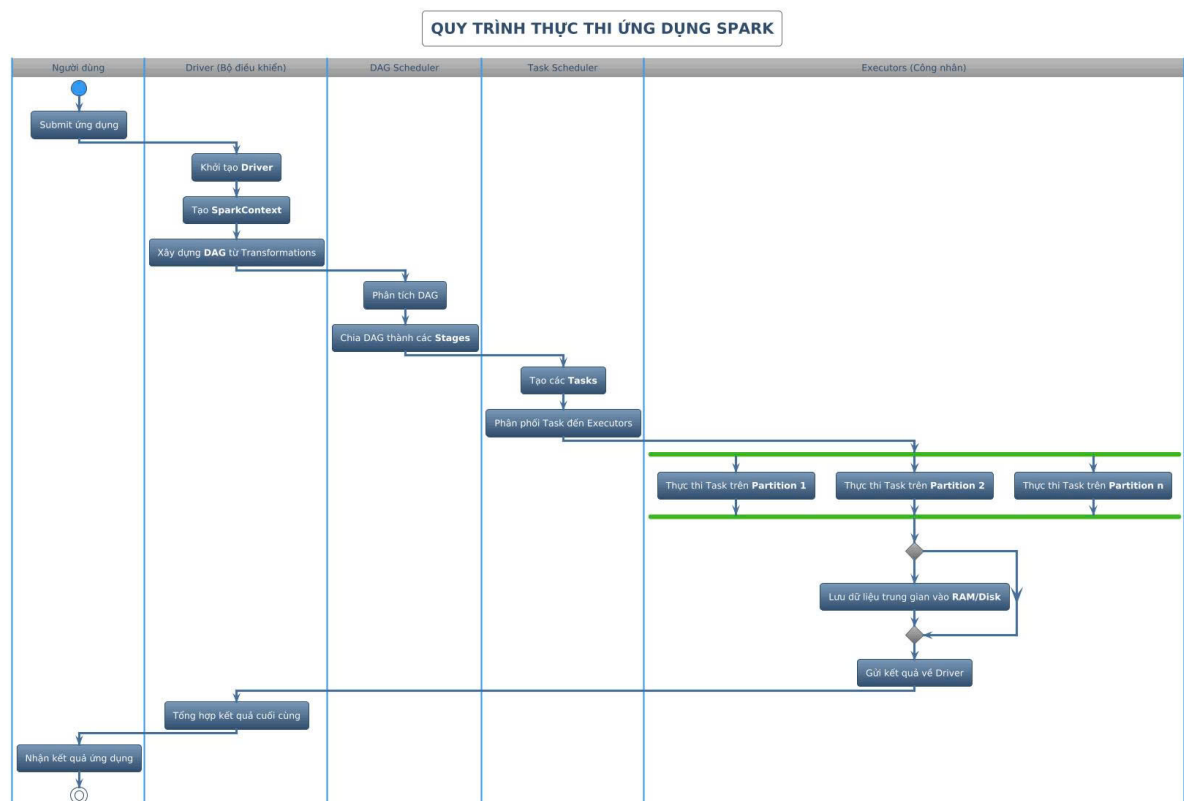
Các stage được thực thi tuần tự theo thứ tự phụ thuộc, nhưng các task trong cùng một stage được thực thi song song.

Luồng thực thi tổng thể

Quá trình thực thi một ứng dụng Apache Spark bắt đầu khi người dùng submit chương trình, lúc này Driver Program được khởi tạo và đóng vai trò điều phối toàn bộ hệ

thống. Driver tiến hành tạo SparkContext (hoặc SparkSession) để thiết lập kết nối với cluster và đồng thời xây dựng Directed Acyclic Graph (DAG) dựa trên các phép biến đổi (transformations) được định nghĩa trong chương trình. Tiếp theo, DAG Scheduler phân tích DAG này và chia nhỏ thành các stage dựa trên mối quan hệ phụ thuộc dữ liệu, đặc biệt là phân biệt giữa các phép biến đổi dạng narrow và wide. Sau đó, Task Scheduler tiếp tục phân rã mỗi stage thành nhiều task tương ứng với các partition dữ liệu và phân phối các task này đến các Executors trên các worker nodes.

Trong quá trình thực thi, các Executors xử lý dữ liệu song song trên từng partition, đồng thời có thể lưu trữ dữ liệu trung gian trong bộ nhớ nhằm tái sử dụng cho các phép toán tiếp theo, giúp giảm chi phí truy xuất dữ liệu và cải thiện hiệu năng. Sau khi hoàn thành các task, Executors gửi kết quả và trạng thái thực thi về Driver. Cuối cùng, Driver tổng hợp toàn bộ kết quả từ các Executors và trả về cho người dùng hoặc ghi ra hệ thống lưu trữ, hoàn tất quá trình xử lý của một ứng dụng Spark.



Hình 2: Quy trình thực thi Spark

2.2.3 So sánh Apache Spark với Hadoop MapReduce

Apache Spark và Hadoop MapReduce là hai framework xử lý dữ liệu lớn phổ biến, đều hỗ trợ xử lý phân tán trên cụm máy tính. Tuy nhiên, chúng có sự khác biệt đáng kể về kiến trúc, mô hình xử lý và hiệu năng.

Bảng 3: So sánh Apache Spark và Hadoop MapReduce

Tiêu chí	Hadoop MapReduce	Apache Spark
Mô hình xử lý	Map → Reduce (2 giai đoạn cố định)	DAG (Directed Acyclic Graph), đa giai đoạn linh hoạt
Cách xử lý dữ liệu	Dựa trên đĩa (disk-based)	Trong bộ nhớ (in-memory)
Hiệu năng	Chậm do phải đọc/ghi đĩa nhiều lần	Nhanh hơn (có thể nhanh hơn 10–100 lần)
Xử lý dữ liệu trung gian	Lưu trên HDFS (đĩa)	Lưu trong RAM (RDD, DataFrame)
Hỗ trợ xử lý	Batch processing	Batch, Streaming, Machine Learning, Graph
Xử lý lặp (Iterative)	Kém hiệu quả	Rất hiệu quả (tái sử dụng dữ liệu trong RAM)
Độ trễ (Latency)	Cao	Thấp
Khả năng mở rộng	Tốt, phù hợp hệ thống lớn	Tốt, nhưng cần tài nguyên mạnh hơn
Yêu cầu phần cứng	Thấp (ít RAM)	Cao (cần nhiều RAM)
Độ phức tạp lập trình	Cao, API thấp	Thấp hơn, API cao (Python, Scala, R, Java)
Thư viện tích hợp	Ít, phải dùng thêm công cụ khác	Nhiều (Spark SQL, MLlib, Streaming, GraphX)
Khả năng tối ưu hóa	Hạn chế	Tối ưu tự động qua DAG Scheduler
Use case phù hợp	Xử lý batch lớn, không yêu cầu thời gian thực	Phân tích dữ liệu nhanh, AI/ML, real-time

Bảng 3 cho thấy Apache Spark vượt trội hơn Hadoop MapReduce về hiệu năng và tính linh hoạt nhờ khả năng xử lý dữ liệu trong bộ nhớ và mô hình DAG. Tuy nhiên, Hadoop MapReduce vẫn có lợi thế về chi phí và phù hợp với các bài toán xử lý dữ liệu theo lô quy mô lớn không yêu cầu thời gian thực.

2.2.4 Các thành phần quan trọng trong Spark

Apache Spark được thiết kế như một hệ sinh thái xử lý dữ liệu lớn thống nhất, bao gồm nhiều thành phần tích hợp chặt chẽ nhằm hỗ trợ đa dạng các bài toán từ xử lý dữ liệu truyền thống đến phân tích nâng cao và học máy. Kiến trúc của Spark mang tính

mô-đun, trong đó mỗi thành phần đảm nhiệm một vai trò cụ thể nhưng vẫn có khả năng tương tác linh hoạt với nhau. Các thành phần chính của Spark bao gồm:

Spark Core:

Spark Core là nền tảng cốt lõi của hệ sinh thái Spark, chịu trách nhiệm quản lý tài nguyên, lập lịch tác vụ và xử lý dữ liệu phân tán. Thành phần này cung cấp các API cơ bản và các cấu trúc dữ liệu chính như:

- **RDD (Resilient Distributed Dataset):** Là cấu trúc dữ liệu phân tán bất biến, cho phép xử lý song song trên nhiều node. RDD hỗ trợ cơ chế fault tolerance thông qua lineage (lịch sử biến đổi dữ liệu).
- **DataFrame:** Là cấu trúc dữ liệu dạng bảng có schema, tương tự bảng trong cơ sở dữ liệu hoặc DataFrame trong Pandas, giúp tối ưu hóa truy vấn và thao tác dữ liệu.
- **Dataset:** Là sự kết hợp giữa RDD và DataFrame, cung cấp kiểu dữ liệu an toàn (type-safe) và tối ưu hóa hiệu năng.

Spark Core đóng vai trò nền tảng cho tất cả các thành phần khác của Spark.

Spark SQL (Xử lý dữ liệu có cấu trúc)

Spark SQL là module cho phép xử lý dữ liệu có cấu trúc (structured data) và bán cấu trúc (semi-structured data) thông qua ngôn ngữ SQL hoặc API DataFrame/Dataset. Các thành phần quan trọng trong Spark SQL:

- **Catalyst Optimizer:** Là bộ tối ưu hóa truy vấn, sử dụng các kỹ thuật như rule-based và cost-based optimization để cải thiện hiệu năng thực thi.
- **Tungsten Execution Engine:** Tối ưu hóa việc sử dụng bộ nhớ và CPU thông qua việc quản lý bộ nhớ ngoài JVM và tối ưu hóa mã máy.

Spark SQL là một thành phần quan trọng của Apache Spark, cung cấp khả năng xử lý và phân tích dữ liệu lớn thông qua ngôn ngữ truy vấn SQL quen thuộc. Công cụ này cho phép người dùng thực hiện các truy vấn SQL trên tập dữ liệu có quy mô lớn một cách hiệu quả, đồng thời hỗ trợ nhiều định dạng dữ liệu phổ biến như JSON, Parquet, ORC và Avro, giúp tăng tính linh hoạt trong việc tích hợp và xử lý dữ liệu từ nhiều nguồn khác nhau. Bên cạnh đó, Spark SQL có khả năng tích hợp với Hive thông qua Hive Metastore, cho phép tái sử dụng các bảng và metadata sẵn có trong hệ sinh thái Hadoop. Đặc biệt, Spark SQL còn được trang bị bộ tối ưu hóa truy vấn (Catalyst Optimizer), giúp tự động tối ưu các truy vấn phức tạp nhằm nâng cao hiệu năng xử lý. Nhờ những đặc điểm này, Spark SQL góp phần thu hẹp khoảng cách giữa các hệ quản trị cơ sở dữ liệu truyền thống và các hệ thống xử lý Big Data hiện đại.

Spark Streaming (Xử lý dữ liệu thời gian thực)

Spark Streaming mở rộng khả năng của Spark sang xử lý dữ liệu thời gian thực (real-time processing). Thay vì xử lý từng record riêng lẻ, Spark Streaming sử dụng mô hình

micro-batch, trong đó dữ liệu được chia thành các batch nhỏ theo thời gian (ví dụ: mỗi 1–2 giây).

Spark Streaming là một thành phần của Apache Spark cho phép xử lý dữ liệu dạng dòng (streaming data) với độ trễ thấp, phù hợp cho các ứng dụng yêu cầu phản hồi gần thời gian thực. Hệ thống có khả năng tiếp nhận dữ liệu từ nhiều nguồn khác nhau như Apache Kafka, Flume, socket hay HDFS, từ đó thực hiện xử lý liên tục trên các luồng dữ liệu. Spark Streaming hỗ trợ nhiều phép toán phổ biến như map, reduce và window operations, giúp người dùng dễ dàng xây dựng các pipeline xử lý dữ liệu theo thời gian. Bên cạnh đó, Spark còn cung cấp Structured Streaming, một phiên bản nâng cao cho phép xử lý dữ liệu streaming dựa trên mô hình DataFrame/Dataset với API thống nhất, giúp đơn giản hóa việc phát triển và tối ưu hệ thống. Nhờ những đặc điểm này, Spark Streaming đặc biệt phù hợp với các bài toán yêu cầu xử lý nhanh như giám sát hệ thống (monitoring), phân tích log hoặc xử lý dữ liệu từ các thiết bị IoT.

Spark GraphX (Xử lý và phân tích đồ thị)

GraphX là thư viện xử lý đồ thị của Spark, cho phép biểu diễn và phân tích dữ liệu dạng đồ thị trong môi trường phân tán.

GraphX hỗ trợ các thao tác trên đồ thị như xây dựng, biến đổi và tính toán trên các đỉnh và cạnh. Ngoài ra, nó còn cung cấp nhiều thuật toán đồ thị phổ biến như PageRank và Connected Components.

Thư viện này được ứng dụng rộng rãi trong các lĩnh vực như phân tích mạng xã hội, hệ thống gợi ý và phân tích quan hệ giữa các đối tượng trong dữ liệu lớn.

MLlib (Machine Learning trên Spark)

MLlib là thư viện học máy của Spark, được thiết kế để hỗ trợ xây dựng và triển khai các mô hình machine learning trên dữ liệu lớn.

MLlib cung cấp nhiều thuật toán như phân loại, hồi quy, phân cụm và giảm chiều dữ liệu. Bên cạnh đó, thư viện còn hỗ trợ xây dựng pipeline hoàn chỉnh cho quá trình học máy, bao gồm tiền xử lý dữ liệu, huấn luyện mô hình và đánh giá kết quả.

Ưu điểm nổi bật của MLlib là khả năng xử lý dữ liệu phân tán, giúp rút ngắn thời gian huấn luyện mô hình trên các tập dữ liệu lớn.

Tổng kết

Các thành phần của Apache Spark được thiết kế theo hướng tích hợp và bổ trợ lẫn nhau, tạo thành một hệ sinh thái hoàn chỉnh cho xử lý dữ liệu lớn. Trong đó, Spark Core đóng vai trò nền tảng, còn các thành phần như Spark SQL, Spark Streaming, GraphX và MLlib giúp mở rộng khả năng của hệ thống sang nhiều lĩnh vực khác nhau. Nhờ đó, Apache Spark trở thành một công cụ mạnh mẽ và linh hoạt, đáp ứng hiệu quả các yêu cầu xử lý dữ liệu hiện đại.

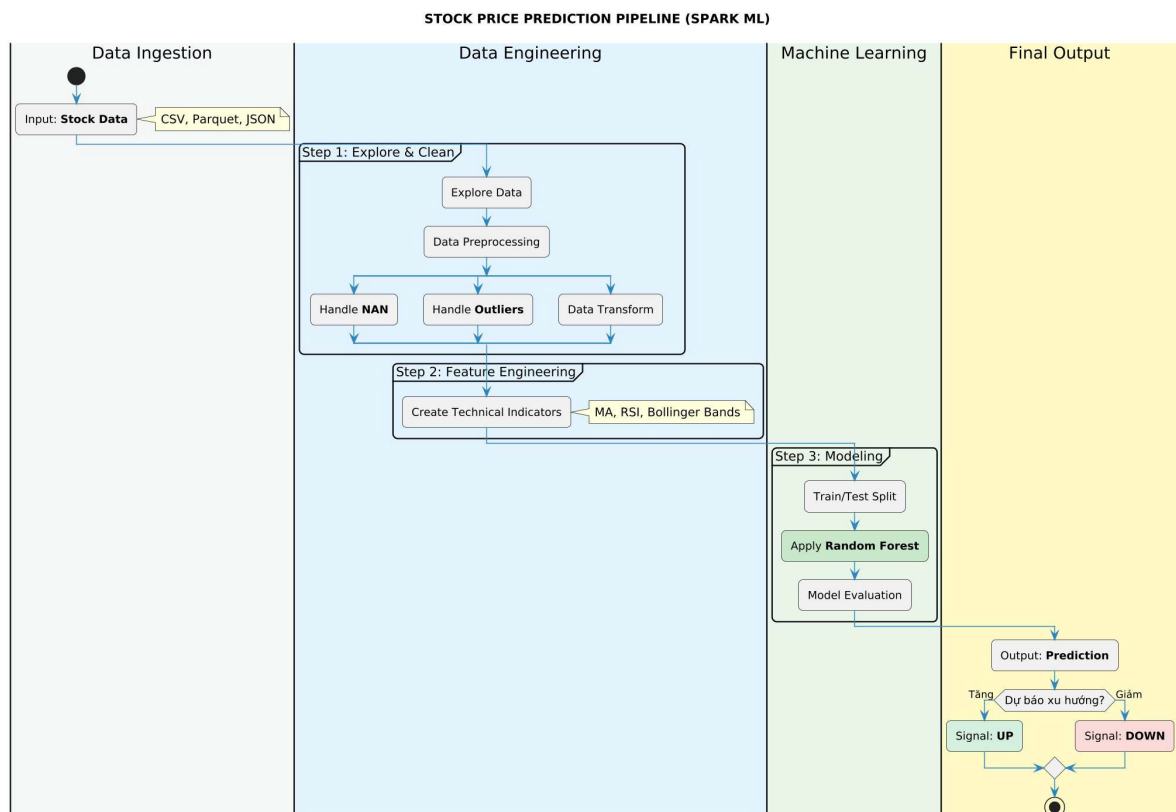
CHƯƠNG 3: PHƯƠNG PHÁP THỰC NGHIỆM

3.1 Tổng quan phương pháp

Pipeline dự báo giá cổ phiếu được thiết kế theo một luồng xử lý liên tục và logic từ dữ liệu thô đến kết quả dự đoán. Đầu tiên, dữ liệu được thu thập từ nhiều nguồn với các định dạng khác nhau như CSV, Parquet và JSON (Data Ingestion), đảm bảo tính đa dạng và đầy đủ của thông tin đầu vào. Sau đó, dữ liệu được chuyển sang giai đoạn Data Engineering, nơi tiến hành khám phá và tiền xử lý nhằm nâng cao chất lượng dữ liệu; các bước này bao gồm làm sạch, xử lý giá trị thiếu, loại bỏ nhiễu (outliers) và biến đổi dữ liệu về dạng phù hợp.

Tiếp nối, dữ liệu đã được làm sạch được sử dụng trong bước Feature Engineering để xây dựng các đặc trưng quan trọng, đặc biệt là các chỉ báo kỹ thuật như MA, RSI và Bollinger Bands, giúp mô hình nắm bắt tốt hơn xu hướng thị trường. Trên cơ sở đó, dữ liệu được đưa vào giai đoạn Machine Learning, trong đó thực hiện chia tập train/test, huấn luyện mô hình Random Forest và đánh giá hiệu quả mô hình một cách có hệ thống.

Cuối cùng, từ mô hình đã huấn luyện, hệ thống tạo ra dự báo xu hướng giá cổ phiếu trong tương lai, biểu diễn dưới dạng tín hiệu giao dịch (UP hoặc DOWN). Toàn bộ pipeline đảm bảo sự liên kết chặt chẽ giữa các bước, giúp chuyển đổi dữ liệu thô thành thông tin có giá trị phục vụ cho việc ra quyết định.



Hình 3: Tổng quan phương pháp nghiên cứu

3.2 Mô tả bộ dữ liệu (Dataset)

3.2.1 Nguồn dữ liệu

Trong đề án này, bộ dữ liệu được sử dụng là 9000+ Tickers of Stock Market Data (Full History), được công bố trên nền tảng Kaggle [21]. Bộ dữ liệu cung cấp thông tin lịch sử của thị trường chứng khoán với hơn 9.000 mã cổ phiếu (tickers), bao phủ khoảng thời gian dài từ năm 1962 đến năm 2024. Dữ liệu được tổng hợp từ các nguồn tài chính đáng tin cậy và được lưu trữ dưới dạng tệp CSV/Parquet, với kích thước lớn (khoảng vài GB), phù hợp cho các bài toán Big Data và xử lý phân tán với Apache Spark.

3.2.2 Đặc trưng dữ liệu

Bộ dữ liệu được tổ chức theo dạng bảng (tabular data), trong đó mỗi dòng tương ứng với thông tin giao dịch của một cổ phiếu tại một thời điểm (theo ngày). Các thuộc tính chính của dữ liệu bao gồm:

Bảng 4: Mô tả các thuộc tính dữ liệu cổ phiếu

Thuộc tính	Mô tả
Date	Ngày giao dịch
Ticker	Mã cổ phiếu
Open	Giá mở cửa
High	Giá cao nhất trong ngày
Low	Giá thấp nhất trong ngày
Close	Giá đóng cửa
Volume	Khối lượng giao dịch
Dividends	Cổ tức (nếu có)
Stock Splits	Thông tin chia tách cổ phiếu

Các đặc trưng này thuộc nhóm dữ liệu OHLCV (Open, High, Low, Close, Volume) – là dạng dữ liệu tiêu chuẩn trong phân tích tài chính và dự báo giá cổ phiếu.

3.2.3 Quy mô và phạm vi dữ liệu

Bộ dữ liệu có quy mô lớn và mang tính chất chuỗi thời gian dài hạn với các đặc điểm sau:

- Số lượng mã cổ phiếu: hơn 9.000.
- Khoảng thời gian: từ năm 1962 đến năm 2024.
- Tần suất dữ liệu: theo ngày (daily time series).

- Kích thước dữ liệu: 3.5GB (CSV).

Với quy mô lớn và thời gian quan sát dài, bộ dữ liệu này phù hợp cho các bài toán phân tích dữ liệu lớn, đặc biệt là các ứng dụng sử dụng Apache Spark để xử lý và khai thác dữ liệu hiệu quả.

3.3 Tiền xử lý dữ liệu (Data Preprocessing)

3.3.1 Phân tích dữ liệu ban đầu (EDA)

Mục tiêu của bước phân tích dữ liệu ban đầu (EDA) là nhằm khám phá đặc điểm của bộ dữ liệu, phát hiện các vấn đề tiềm ẩn và đưa ra định hướng cho các bước tiền xử lý và xây dựng mô hình ở các phần tiếp theo. Việc thực hiện EDA giúp hiểu rõ hơn về cấu trúc, phân bố và mối quan hệ giữa các biến trong dữ liệu.

Bảng 5: Các bước phân tích dữ liệu

Thành phần phân tích	Nội dung thực hiện	Mục đích
Thống kê mô tả dữ liệu	Sử dụng các chỉ số thống kê cơ bản như số lượng bản ghi, số lượng biến, giá trị trung bình, trung vị và độ lệch chuẩn	Cung cấp cái nhìn tổng quan về dữ liệu, đánh giá mức độ biến động và làm cơ sở cho các phân tích chi tiết hơn
Phân tích dữ liệu thiếu	Xác định sự tồn tại và phân bố của các giá trị khuyết trong từng thuộc tính	Đánh giá mức độ hoàn chỉnh của dữ liệu và định hướng lựa chọn phương pháp xử lý phù hợp nhằm đảm bảo chất lượng dữ liệu
Phân tích phân bố dữ liệu	Sử dụng các biểu đồ như histogram, density plot và boxplot để quan sát phân phối dữ liệu	Xác định hình dạng phân phối, độ lệch và outliers, từ đó định hướng các kỹ thuật tiền xử lý như chuẩn hóa hoặc biến đổi dữ liệu
Phân tích xu hướng theo thời gian	Phân tích sự biến động của dữ liệu theo thời gian thông qua các biểu đồ chuỗi thời gian	Nhận diện xu hướng và đặc điểm động của dữ liệu, hỗ trợ lựa chọn cách biểu diễn và mô hình phù hợp

Thành phần phân tích	Nội dung thực hiện	Mục đích
Phân tích tương quan giữa các biến	Đánh giá mối quan hệ giữa các biến thông qua ma trận tương quan	Phát hiện các biến có liên hệ mạnh hoặc trùng lặp thông tin, hỗ trợ lựa chọn đặc trưng và giảm đa cộng tuyến

3.3.2 Làm sạch dữ liệu (Data Cleaning)

Làm sạch dữ liệu là bước quan trọng trong quá trình tiền xử lý nhằm đảm bảo chất lượng và độ tin cậy của dữ liệu đầu vào. Mục đích của bước này là phát hiện và xử lý các vấn đề tồn tại trong dữ liệu như giá trị thiếu (missing values), dữ liệu trùng lặp (duplicate records) hoặc các giá trị không hợp lệ.

Việc xử lý dữ liệu thiếu giúp tránh làm sai lệch kết quả phân tích và cải thiện hiệu suất của mô hình. Đồng thời, loại bỏ các bản ghi trùng lặp góp phần đảm bảo tính nhất quán và chính xác của dữ liệu. Ngoài ra, việc kiểm tra và xử lý các giá trị bất thường hoặc không hợp lệ giúp hạn chế nhiễu và giảm ảnh hưởng tiêu cực đến quá trình huấn luyện mô hình. Tóm lại, làm sạch dữ liệu giúp chuẩn hóa dữ liệu đầu vào, tạo nền tảng vững chắc cho các bước xử lý tiếp theo và nâng cao hiệu quả của toàn bộ hệ thống phân tích và dự đoán.

3.3.3 Xử lý giá trị ngoại lai (Outlier Handling)

Xử lý giá trị ngoại lai nhằm phát hiện và đánh giá các điểm dữ liệu có giá trị bất thường so với phần lớn dữ liệu còn lại. Mục đích của bước này là giảm thiểu ảnh hưởng tiêu cực của các giá trị cực đoan đến quá trình phân tích và huấn luyện mô hình.

Việc xem xét và xử lý outliers giúp cải thiện độ ổn định của dữ liệu, hạn chế hiện tượng sai lệch trong quá trình học của mô hình. Tùy theo đặc điểm của bài toán, các giá trị ngoại lai có thể được giữ lại nếu chúng phản ánh đúng bản chất của dữ liệu, hoặc được điều chỉnh để tránh gây nhiễu.

3.3.4 Biến đổi dữ liệu (Data Transformation)

Biến đổi dữ liệu được thực hiện nhằm đưa dữ liệu về dạng phù hợp hơn cho quá trình học của mô hình. Mục đích của bước này là cải thiện đặc tính thống kê của dữ liệu, giảm độ lệch phân phối và đảm bảo các biến có cùng thang đo.

Các phép biến đổi như chuẩn hóa hoặc biến đổi log giúp dữ liệu trở nên ổn định hơn, từ đó hỗ trợ các thuật toán học máy hoạt động hiệu quả hơn. Đồng thời, bước này còn giúp giảm ảnh hưởng của các giá trị lớn và cải thiện khả năng hội tụ của mô hình.

3.3.5 *Xây dựng đặc trưng*

Xây dựng đặc trưng nhằm tạo ra các biến mới có ý nghĩa hơn từ dữ liệu ban đầu, giúp mô hình khai thác tốt hơn thông tin tiềm ẩn trong dữ liệu. Mục đích của bước này là nâng cao khả năng biểu diễn dữ liệu và cải thiện hiệu suất dự đoán của mô hình.

Việc tạo thêm các đặc trưng phù hợp giúp phản ánh rõ hơn bản chất của dữ liệu, đồng thời hỗ trợ mô hình học được các mối quan hệ phức tạp mà các biến ban đầu có thể chưa thể hiện đầy đủ.

3.4 **Xây dựng và huấn luyện mô hình (Modeling)**

3.4.1 *Chuẩn bị dữ liệu cho mô hình*

Trước khi huấn luyện, dữ liệu được tổ chức lại theo định dạng phù hợp với các thuật toán học máy. Mục đích của bước này là đảm bảo dữ liệu đầu vào có cấu trúc nhất quán và sẵn sàng cho quá trình học. Ngoài ra, dữ liệu được phân chia thành các tập huấn luyện và kiểm tra nhằm đánh giá khả năng tổng quát hóa của mô hình, tránh hiện tượng quá khớp.

3.4.2 *Lựa chọn mô hình*

Trong nghiên cứu này, mô hình Random Forest được lựa chọn nhằm khai thác hiệu quả các đặc trưng của dữ liệu và cải thiện khả năng dự đoán. Random Forest là một phương pháp học máy thuộc nhóm ensemble learning, hoạt động bằng cách kết hợp nhiều cây quyết định để đưa ra kết quả cuối cùng.

Việc lựa chọn Random Forest xuất phát từ khả năng xử lý tốt dữ liệu có tính phi tuyến và chứa nhiều nhiễu. Bên cạnh đó, mô hình này có khả năng giảm hiện tượng quá khớp nhờ cơ chế tổng hợp từ nhiều mô hình con, từ đó nâng cao độ ổn định và khả năng tổng quát hóa.

Ngoài ra, Random Forest không yêu cầu dữ liệu phải tuân theo các giả định phân phối cụ thể, phù hợp với đặc điểm của dữ liệu tài chính thường có phân phối phức tạp và biến động mạnh. Mô hình cũng có khả năng xử lý tốt tập dữ liệu có số lượng lớn đặc trưng và hỗ trợ đánh giá mức độ quan trọng của từng biến, góp phần nâng cao khả năng giải thích của mô hình.

Tóm lại, Random Forest được lựa chọn do tính linh hoạt, khả năng xử lý dữ liệu hiệu quả và phù hợp với bài toán dự đoán trên dữ liệu lớn trong môi trường Apache Spark.

3.4.3 *Huấn luyện mô hình*

Huấn luyện mô hình là quá trình sử dụng dữ liệu đã được tiền xử lý để học ra các tham số tối ưu, từ đó mô hình có thể nắm bắt được các mối quan hệ tiềm ẩn giữa các đặc

trưng và biến mục tiêu. Mục đích của bước này là xây dựng một mô hình có khả năng dự đoán chính xác và tổng quát hóa tốt trên dữ liệu chưa từng thấy.

Trong nghiên cứu này, quá trình huấn luyện được thực hiện trên nền tảng Apache Spark thông qua thư viện Spark MLlib, cho phép xử lý dữ liệu ở quy mô lớn và tận dụng khả năng tính toán phân tán. Việc sử dụng Spark MLlib giúp tăng tốc quá trình huấn luyện, đồng thời đảm bảo khả năng mở rộng khi làm việc với dữ liệu lớn.

Mô hình Random Forest được huấn luyện trên tập dữ liệu huấn luyện đã được chuẩn bị trước đó. Trong quá trình này, dữ liệu được phân phối trên nhiều nút tính toán, giúp thực hiện song song các phép tính và rút ngắn thời gian xử lý. Ngoài ra, việc huấn luyện được thực hiện với các cấu hình khác nhau nhằm đảm bảo mô hình có thể học được các đặc trưng quan trọng của dữ liệu.

3.4.4 Đánh giá mô hình

Đánh giá mô hình nhằm đo lường hiệu quả hoạt động của mô hình trên dữ liệu chưa từng được sử dụng trong quá trình huấn luyện. Mục đích của bước này là kiểm tra khả năng tổng quát hóa và độ tin cậy của mô hình. Các chỉ số đánh giá được sử dụng để so sánh giữa các mô hình và lựa chọn phương pháp phù hợp nhất cho bài toán.

CHƯƠNG 4: KẾT QUẢ THỰC NGHIỆM

4.1 Kết quả phân tích dữ liệu

4.1.1 Thống kê mô tả dữ liệu

Trong bước này, bộ dữ liệu được phân tích thông qua các chỉ số thống kê cơ bản nhằm cung cấp cái nhìn tổng quan về quy mô, cấu trúc và đặc điểm của dữ liệu.

a) Quy mô bộ dữ liệu

Bảng 6: Thống kê tổng quan dữ liệu

Chỉ tiêu	Giá trị
Tổng số dòng	34,646,258
Tổng số cột	9
Khoảng thời gian	1962-01-02 → 2024-11-04
Số lượng mã cổ phiếu	9,315

Kết quả cho thấy bộ dữ liệu có quy mô rất lớn với hơn 34 triệu bản ghi, bao phủ hơn 60 năm giao dịch của 9,315 mã cổ phiếu. Điều này phản ánh tính đa dạng và đầy đủ của dữ liệu, đồng thời phù hợp với các bài toán phân tích dữ liệu lớn.

b) Cấu trúc dữ liệu

Bảng 7: Cấu trúc schema dữ liệu

Tên cột	Kiểu dữ liệu
Date	Date
Ticker	String
Open	Double
High	Double
Low	Double
Close	Double
Volume	Double
Dividends	Double
Stock Splits	Double

Cấu trúc dữ liệu bao gồm các biến tài chính cơ bản thuộc nhóm OHLCV, cho phép mô tả đầy đủ biến động giá và khối lượng giao dịch của cổ phiếu theo thời gian.

c) Thống kê các biến số

Bảng 8: Thống kê mô tả các biến giá và khối lượng

Chỉ số	Open	High	Low	Close	Volume
Count	34,646,149	34,646,149	34,646,149	34,646,152	34,646,258
Mean	$\sim 1.115 \times 10^{23}$	$\sim 1.115 \times 10^{23}$	$\sim 1.115 \times 10^{23}$	$\sim 1.115 \times 10^{23}$	1,339,229.87
Min	$\sim -8.21 \times 10^?$	$\sim -8.21 \times 10^?$	$\sim -8.21 \times 10^?$	$\sim -8.21 \times 10^?$	0
Max	$\sim 1.507 \times 10^?$	$\sim 1.507 \times 10^?$	$\sim 1.507 \times 10^?$	$\sim 1.507 \times 10^?$	9.23×10^9

Kết quả thống kê cho thấy các biến giá (Open, High, Low, Close) có giá trị trung bình và khoảng biến thiên rất lớn, cho thấy dữ liệu tồn tại sự phân tán mạnh và có thể chứa các giá trị bất thường. Trong khi đó, biến Volume có sự chênh lệch đáng kể giữa giá trị nhỏ nhất và lớn nhất, phản ánh sự khác biệt lớn về khối lượng giao dịch giữa các cổ phiếu.

Ngoài ra, số lượng quan sát giữa các biến có sự khác biệt nhỏ, cho thấy vẫn tồn tại một lượng dữ liệu thiếu nhất định trong bộ dữ liệu.

4.1.2 Phân tích dữ liệu thiếu

Kết quả kiểm tra dữ liệu thiếu cho thấy hầu hết các thuộc tính trong bộ dữ liệu đều không tồn tại giá trị khuyết, bao gồm Date, Ticker, Volume, Dividends và Stock Splits. Tuy nhiên, vẫn ghi nhận một số giá trị thiếu ở nhóm biến giá. Cụ thể, biến Open, High, Low và Close đều thiếu 106 giá trị trên tổng số hơn 34 triệu quan sát.

Nhìn chung, tỷ lệ dữ liệu thiếu là rất nhỏ so với toàn bộ tập dữ liệu, cho thấy bộ dữ liệu có chất lượng cao và mức độ hoàn chỉnh tốt. Mặc dù vậy, các giá trị thiếu này vẫn cần được xử lý trong giai đoạn tiền xử lý nhằm đảm bảo tính nhất quán của dữ liệu và tránh ảnh hưởng đến độ chính xác của các bước phân tích và mô hình hóa tiếp theo.

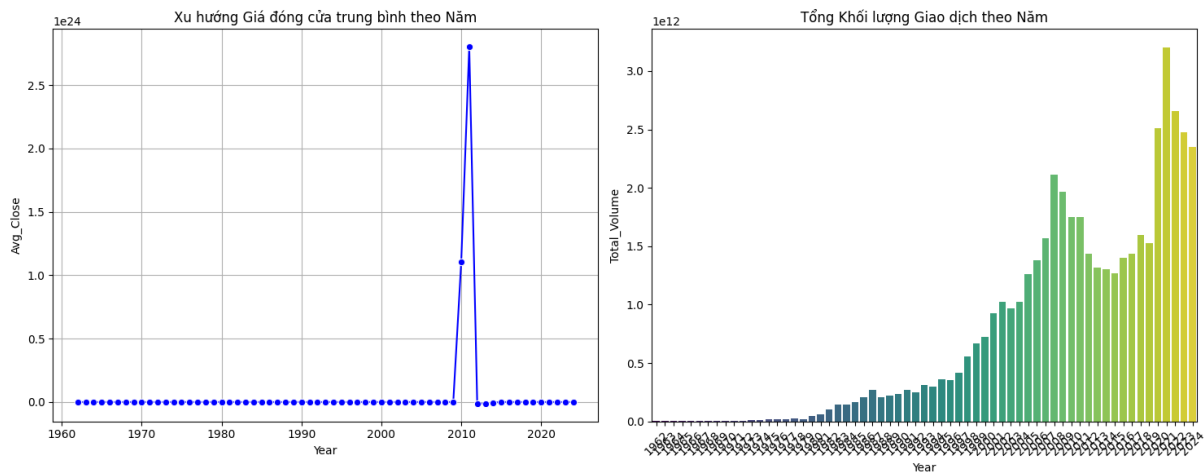
4.1.3 Phân tích phân bố dữ liệu và xu hướng theo thời gian

a) Phân tích xu hướng biến động lịch sử

Tiến hành quan sát dữ liệu trên phương diện thời gian để hiểu rõ bối cảnh lịch sử của bộ dữ liệu. Kết quả cho thấy một sự tương phản rõ rệt: trong khi giá đóng cửa trung bình duy trì mức ổn định qua nhiều thập kỷ thì bất ngờ xuất hiện một điểm đột biến (spike) cực lớn vào khoảng năm 2010. Điều này đặt ra nghi vấn về sai số trong quá trình thu thập hoặc một sự kiện tài chính đặc thù cần phải xử lý kỹ ở bước làm sạch. Ngược lại, khối lượng giao dịch (Volume) cho thấy một sự tăng trưởng bền vững, đặc biệt bùng nổ sau năm 2000, minh chứng cho thấy khối lượng giao dịch ngày càng mạnh mẽ của thị trường chứng khoán hiện đại.

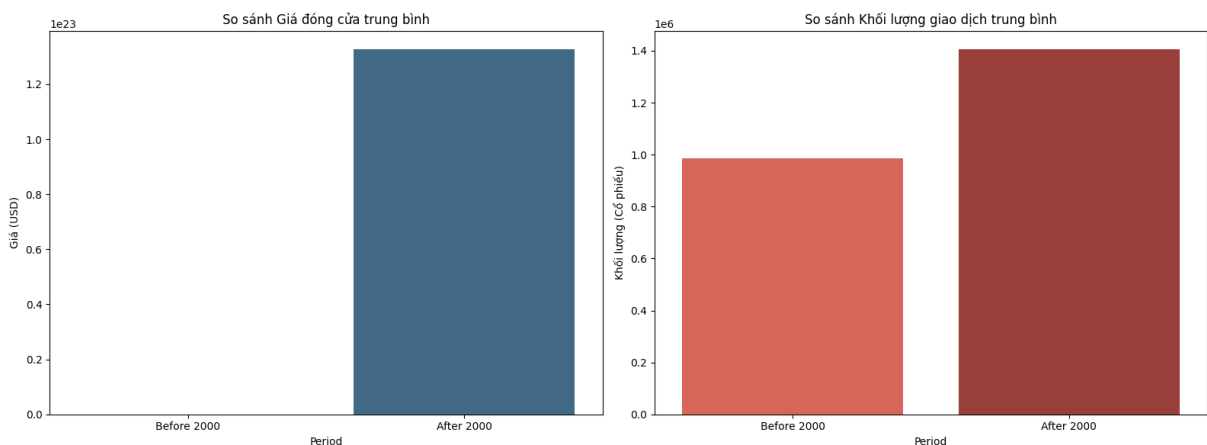
b) So sánh đặc trưng dữ liệu giữa các giai đoạn

Để làm rõ sự khác biệt giữa thị trường truyền thống và thị trường hiện đại, em đã thực hiện chia tách và so sánh dữ liệu tại mốc năm 2000. Qua các biểu đồ cột, có thể thấy giá



Hình 4: Biểu đồ đường thể hiện xu hướng biến động Giá đóng cửa trung bình giai đoạn 1962 - 2024.

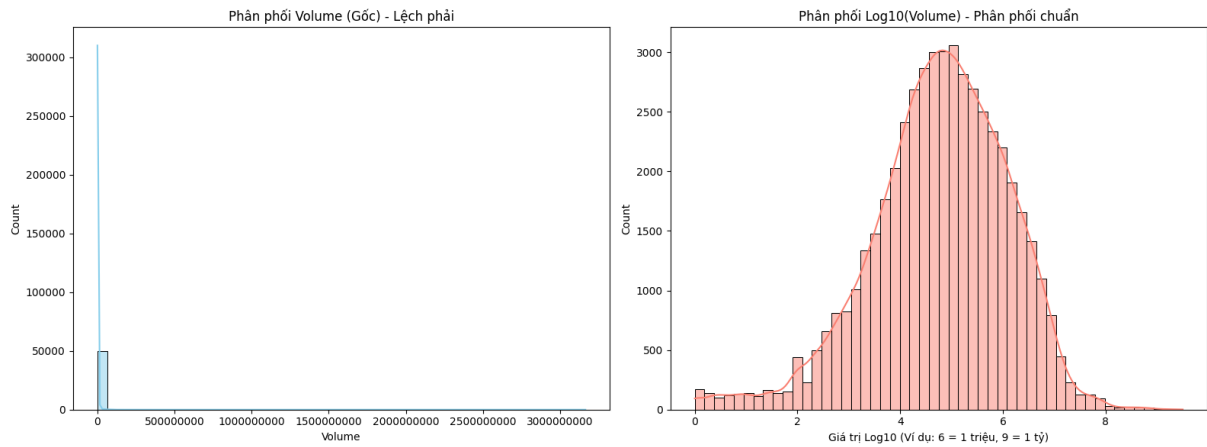
trị cổ phiếu và khối lượng giao dịch sau năm 2000 đều vượt trội hơn hẳn so với giai đoạn trước đó. Cụ thể, khối lượng giao dịch trung bình đã tăng từ mức dưới 1 triệu lên khoảng 1.4 triệu đơn vị mỗi phiên. Sự thay đổi về quy mô này cho thấy dữ liệu từ năm 2000 trở đi mang tính đại diện cao hơn cho các mô hình dự báo tài chính hiện tại, giúp mô hình tránh bị nhiễu bởi các đặc tính lỗi thời của thị trường trong quá khứ.



Hình 5: So sánh giá trị trung bình của Giá và Khối lượng trước và sau năm 2000.

c) Biến đổi dữ liệu và xử lý phân phối lệch

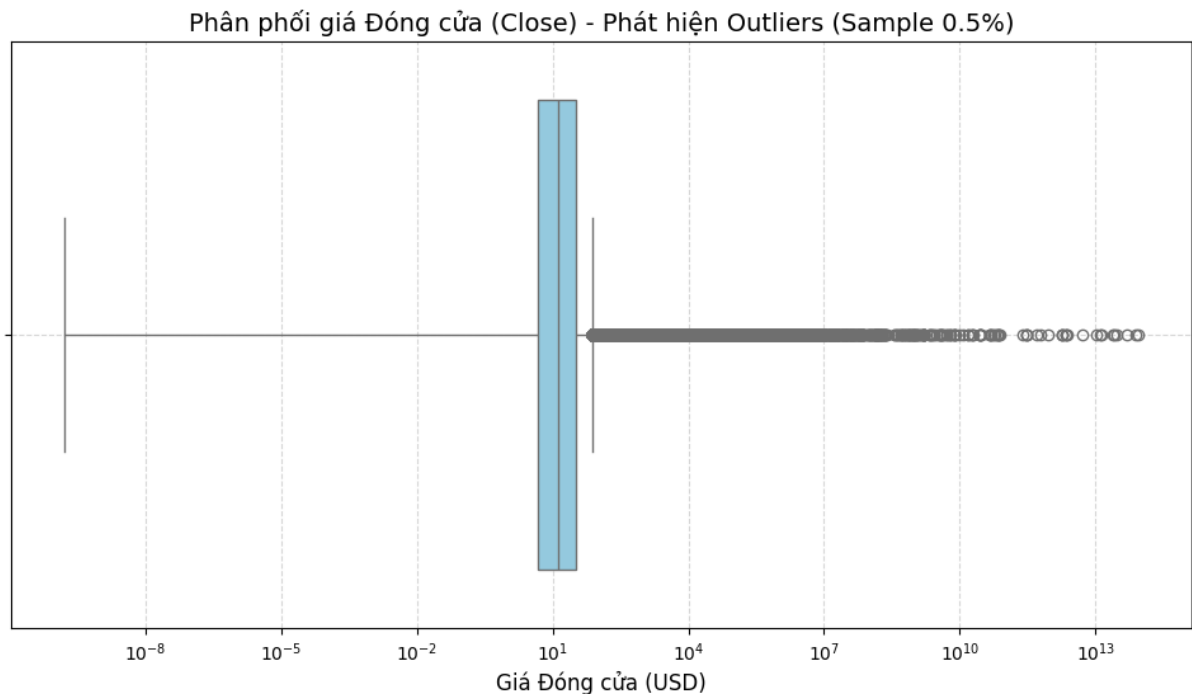
Một trong những thách thức lớn nhất đối với bài toán sử dụng Spark ML là hiện tượng dữ liệu bị lệch (skewness). Qua khảo sát, biến Volume gốc có độ lệch phải cực kỳ nghiêm trọng, điều này có thể gây mất cân bằng và làm giảm hiệu năng của thuật toán Random Forest. Để giải quyết vấn đề này, em đã áp dụng phương pháp biến đổi $\log_{10}$. Kết quả thực nghiệm cho thấy phân phối sau biến đổi đã tiệm cận về dạng phân phối chuẩn (Normal Distribution). Bước tiền xử lý này đóng vai trò quyết định, giúp dữ liệu đầu vào trở nên "mượt" hơn, hỗ trợ mô hình học được các quy luật đặc trưng một cách khách quan nhất.



Hình 6: Phân phối của biến Volume trước và sau khi áp dụng phép biến đổi Log-Scale.

d) Phân tích giá trị ngoại lai

Sử dụng biểu đồ Boxplot kết hợp với thang đo Log để rà soát các giá trị ngoại lai. Hình ảnh phân tích cho thấy sự tồn tại của các điểm dữ liệu nằm xa phạm vi biến động thông thường (lên tới mức 10^{13}), xuất hiện dày đặc ở phía trên của phân phối. Những giá trị này nếu không được xử lý sẽ gây nhiễu cho quá trình phân tách các nhánh trong cây quyết định của Random Forest. Do đó, dựa trên hình ảnh này, em đề xuất sẽ thực hiện kỹ thuật chặn trên (Capping) hoặc loại bỏ các bản ghi không hợp lý để đảm bảo độ tin cậy cho kết quả dự báo "Tăng" hay "Giảm" ở các bước tiếp theo.



Hình 7: Biểu đồ Boxplot nhận diện các giá trị ngoại lai (Outliers) của biến Giá đóng cửa.

4.2 Kết quả tiền xử lý dữ liệu

4.2.1 Loại bỏ các đặc trưng không cần thiết

Trong bước đầu tiên của quá trình làm sạch dữ liệu, tiến hành loại bỏ các cột "Dividends" và "Stock Splits". Hai biến số này không đóng vai trò trực tiếp trong việc cấu thành các chỉ số kỹ thuật chính, việc loại bỏ chúng giúp giảm bớt chiều dữ liệu, từ đó tối ưu hóa bộ nhớ và tốc độ huấn luyện cho hệ thống Spark.

4.2.2 Xử lý giá trị thiếu (*Handling Missing Values*)

Qua kiểm kê, tập dữ liệu có tỉ lệ giá trị thiếu (null) cực kỳ thấp, chỉ khoảng 109 dòng trên tổng số 34 triệu dòng. Với tỉ lệ sai số không đáng kể này, quyết định sử dụng phương pháp Forward Fill (lấy giá trị của ngày trước đó để điền cho ngày hiện tại). Phương pháp này đảm bảo tính liên tục của chuỗi thời gian chứng khoán mà không làm thay đổi đặc tính thống kê chung của toàn bộ tập dữ liệu lớn.

4.2.3 Cắt dữ liệu theo mốc thời gian

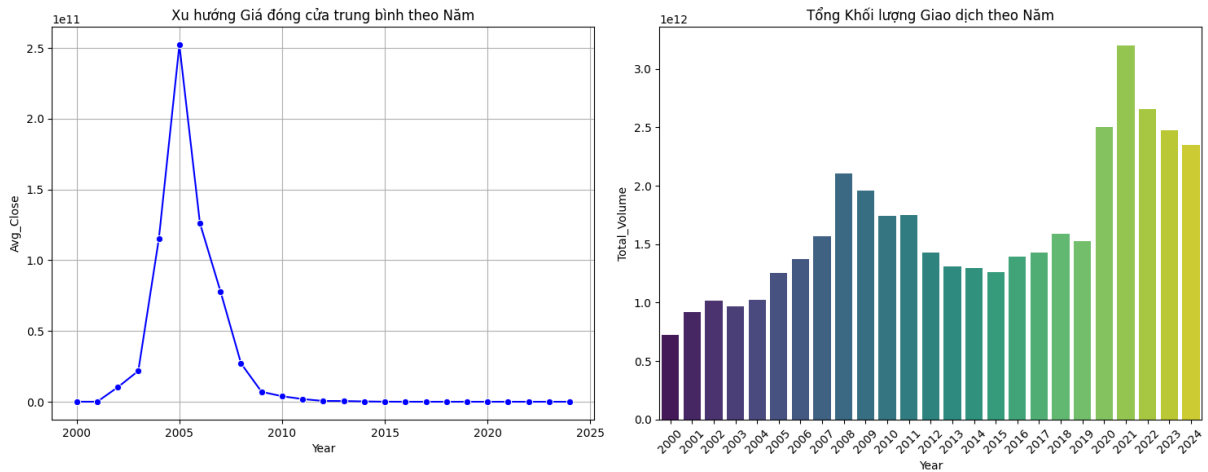
Tập trung phân tích và huấn luyện mô hình dựa trên dữ liệu từ năm 2000 trở đi. Quyết định này dựa trên hai lý do chiến lược:

- Về chất lượng dữ liệu: Giai đoạn này có thanh khoản cực kỳ ổn định, đóng góp hơn 80% tổng khối lượng giao dịch của toàn bộ tập dữ liệu. Điều này giúp mô hình tránh được các nhiễu từ những giai đoạn thị trường sơ khai của thế kỷ trước.
- Về tính thời đại: Thị trường tài chính sau năm 2000 phản ánh rõ nét kỷ nguyên giao dịch điện tử và các thuật toán hiện đại. Việc huấn luyện trên dữ liệu này giúp mô hình Random Forest có khả năng "học" được các quy luật giao dịch sát với thực tế hiện nay nhất.

4.2.4 Xử lý giá trị ngoại lai

Trong quá trình khám phá dữ liệu, em phát hiện các giá trị Open, High, Low, Close có những con số phi thực tế, lên đến hàng triệu USD (mức 10^{15}). Để so sánh, GDP toàn cầu hiện nay cũng chỉ ở mức 10^{14} USD. Do đó, việc tồn tại một cổ phiếu đơn lẻ có mức giá này chắc chắn là những điểm nhiễu cần loại bỏ.

nhận thấy nguyên nhân không đến từ lỗi nhập liệu thủ công. Đây là hệ quả của việc Điều chỉnh giá sau các đợt Gộp cổ phiếu (Reverse Stock Splits). Các công ty (ví dụ như Top Ships Inc.) thực hiện gộp cổ phiếu với tỷ lệ tích lũy cực cao để duy trì niêm yết trên sàn. Khi thuật toán tính toán giá điều chỉnh (Adjusted Price) bằng cách nhân ngược giá trong quá khứ với các tỷ lệ gộp khổng lồ này, nó vô tình tạo ra những con số "ảo" trong lịch sử dữ liệu.



Hình 8: Biểu đồ đường thể hiện xu hướng biến động Giá đóng cửa trung bình giai đoạn 2000 - 2024.

Để đảm bảo tính khách quan và hợp lý cho mô hình dự báo, em thực hiện lọc bỏ toàn bộ các bản ghi có mức giá Đóng cửa (Close) vượt quá 550,000 USD (mức giá tham chiếu cao nhất của cổ phiếu Berkshire Hathaway loại A ngoài thực tế). Việc làm này giúp làm sạch tập huấn luyện, loại bỏ các giá trị ảo do gộp cổ phiếu gây ra, từ đó nâng cao độ tin cậy cho kết quả dự báo của mô hình.

4.2.5 Tạo đặc trưng mới

Sau khi hoàn tất quá trình làm sạch dữ liệu và xử lý các giá trị ngoại lai, nhóm nhận thấy rằng các biến số gốc như giá Open, High, Low, Close đơn thuần chưa đủ để mô hình có thể nắm bắt được những quy luật phức tạp của thị trường chứng khoán. Do đó, nhóm tiến hành xây dựng thêm hai nhóm đặc trưng quan trọng là Sector (Lĩnh vực) và SMA (Đường trung bình động) để gia tăng sức mạnh dự báo cho mô hình.

4.3 Kết quả xây dựng và huấn luyện mô hình

Sau khi xây dựng pipeline xử lý dữ liệu và huấn luyện mô hình Random Forest trên nền tảng PySpark, mô hình được đánh giá bằng phương pháp Walk-forward Validation (Time-series Split) với 3 giai đoạn thời gian khác nhau nhằm đảm bảo tính ổn định và khả năng tổng quát hóa theo thời gian.

4.3.1 Kết quả đánh giá theo từng Fold

Kết quả đánh giá mô hình theo phương pháp Walk-forward Validation được tổng hợp qua ba giai đoạn thời gian khác nhau cho thấy hiệu năng của mô hình tương đối ổn định. Cụ thể, ở Fold 1 (giai đoạn kiểm tra năm 2022), mô hình đạt Accuracy 0.5374 và AUC-ROC 0.5314, với độ chính xác trên hai ngành Financial Services và Technology lần lượt là 0.5382 và 0.5365. Sang Fold 2 (năm 2023), Accuracy giảm nhẹ xuống 0.5358 nhưng

AUC-ROC tăng lên 0.5522, cho thấy khả năng phân biệt lớp có cải thiện; trong đó ngành Technology đạt độ chính xác cao hơn (0.5479) so với Financial Services (0.5244). Đến Fold 3 (năm 2024), mô hình tiếp tục duy trì mức hiệu năng tương đương với Accuracy 0.5320 và AUC-ROC 0.5434, với độ chính xác giữa hai ngành không có sự chênh lệch đáng kể (Financial Services: 0.5298, Technology: 0.5345). Nhìn chung, các chỉ số qua từng fold dao động nhỏ quanh mức 53–55%, phản ánh mô hình có tính ổn định theo thời gian nhưng hiệu năng chưa cao, chỉ nhỉnh hơn mức dự đoán ngẫu nhiên.

Bảng 9: Kết quả đánh giá mô hình theo từng giai đoạn (Walk-forward Validation)

Fold	Giai đoạn Test	Accuracy	AUC-ROC	Financial Services	Technology
Fold 1	2022	0.5374	0.5314	0.5382	0.5365
Fold 2	2023	0.5358	0.5522	0.5244	0.5479
Fold 3	2024	0.5320	0.5434	0.5298	0.5345

4.3.2 Đánh giá tổng thể mô hình

Dựa trên kết quả đánh giá ở tập kiểm tra cuối cùng, mô hình đạt Accuracy 0.5320, Precision 0.5294, Recall 0.5320 và F1-Score 0.4843. Các chỉ số này cho thấy mô hình có hiệu năng ở mức trung bình, chỉ nhỉnh hơn một chút so với dự đoán ngẫu nhiên trong bài toán phân loại nhị phân. Độ chính xác (Accuracy) và Recall gần tương đương nhau, chứng tỏ mô hình không bị thiên lệch rõ rệt về một lớp cụ thể. Đồng thời, Precision cũng ở mức tương đối gần với Accuracy, cho thấy tỷ lệ dự đoán đúng trong các dự đoán dương không quá chênh lệch.

Tuy nhiên, giá trị F1-Score thấp hơn đáng kể so với các chỉ số còn lại phản ánh sự cân bằng giữa Precision và Recall chưa thực sự tốt, đặc biệt trong bối cảnh dữ liệu tài chính thường nhiễu và khó dự đoán. Nhìn chung, mô hình Random Forest đã học được một số quy luật nhất định từ dữ liệu, nhưng khả năng dự đoán xu hướng tăng/giảm giá cổ phiếu vẫn còn hạn chế và chưa đủ mạnh để ứng dụng trong các bài toán thực tế đòi hỏi độ chính xác cao.

Bảng 10: Kết quả đánh giá tổng thể mô hình.

Chỉ số	Giá trị
Accuracy	0.5320
Precision	0.5294
Recall	0.5320
F1-Score	0.4843

4.3.3 Kết quả Backtesting chiến lược giao dịch

Dựa trên kết quả dự đoán của mô hình ở fold cuối cùng (giai đoạn năm 2024), một chiến lược giao dịch đơn giản đã được xây dựng nhằm đánh giá tính ứng dụng thực tế của mô hình. Cụ thể, tại mỗi thời điểm, nếu mô hình dự đoán giá cổ phiếu sẽ tăng (prediction = 1) thì thực hiện mua, ngược lại sẽ không tham gia thị trường. Lợi nhuận chiến lược được tính dựa trên biến động giá thực tế của ngày tiếp theo.

Kết quả backtesting cho thấy chiến lược sử dụng mô hình đạt mức lợi nhuận 127.00%, trong khi chiến lược cơ bản mua và nắm giữ (Buy & Hold) đạt lợi nhuận lên đến 950.56% trong cùng giai đoạn. Điều này dẫn đến giá trị Alpha = -823.56%, cho thấy chiến lược dựa trên mô hình hoạt động kém hiệu quả hơn đáng kể so với thị trường.

Kết quả này phản ánh rằng mặc dù mô hình có khả năng dự đoán xu hướng ở mức nhất định, nhưng độ chính xác chưa đủ cao để chuyển hóa thành lợi nhuận vượt trội trong thực tế. Ngoài ra, chiến lược giao dịch được xây dựng còn đơn giản, chưa xét đến các yếu tố quan trọng như chi phí giao dịch, rủi ro thị trường hay tối ưu danh mục đầu tư. Do đó, để cải thiện hiệu quả, cần kết hợp mô hình với các phương pháp quản lý rủi ro và chiến lược giao dịch nâng cao hơn.

4.3.4 Confusion matrix

Từ ma trận nhầm lẫn (confusion matrix), có thể thấy mô hình đạt được 4353 trường hợp dự đoán đúng lớp giảm (True Negative) và 1051 trường hợp dự đoán đúng lớp tăng (True Positive). Tuy nhiên, số lượng dự đoán sai vẫn còn khá lớn, đặc biệt là 3801 trường hợp False Negative, tức là mô hình dự đoán giá giảm nhưng thực tế lại tăng. Điều này cho thấy mô hình gặp khó khăn trong việc nhận diện các phiên tăng giá. Bên cạnh đó, có 953 trường hợp False Positive, nghĩa là mô hình dự đoán tăng nhưng thực tế lại giảm, tuy không quá lớn nhưng vẫn ảnh hưởng đến độ chính xác chung.

Bảng 11: Confusion Matrix của mô hình Random Forest

	Dự đoán = 0	Dự đoán = 1
Thực tế = 0	4353 (TN)	953 (FP)
Thực tế = 1	3801 (FN)	1051 (TP)

Nhìn tổng thể, mô hình có xu hướng thiên về dự đoán lớp giảm (class 0), thể hiện qua số lượng True Negative cao và False Negative lớn. Điều này dẫn đến việc mô hình bỏ lỡ nhiều cơ hội tăng giá trên thị trường, từ đó làm giảm hiệu quả của chiến lược giao dịch. Kết quả này cũng phù hợp với các chỉ số đã đánh giá trước đó, khi Recall cho lớp tăng thấp và F1-score chưa cao. Do đó, có thể kết luận rằng mô hình chưa học tốt các đặc trưng liên quan đến xu hướng tăng giá và cần được cải thiện thêm để nâng cao khả năng dự đoán.

CHƯƠNG 5: KẾT LUẬN

Trong phạm vi đề tài, nhóm đã tiến hành xây dựng một quy trình hoàn chỉnh cho bài toán phân tích và dự đoán xu hướng giá cổ phiếu dựa trên nền tảng Apache Spark. Cụ thể, nhóm đã thực hiện các bước từ tiền xử lý dữ liệu, xây dựng đặc trưng (feature engineering), thiết kế pipeline, đến huấn luyện và đánh giá mô hình bằng phương pháp Walk-forward Validation phù hợp với dữ liệu chuỗi thời gian.

Kết quả thực nghiệm cho thấy mô hình Random Forest đạt độ chính xác trung bình khoảng 53% và AUC-ROC dao động quanh mức 0.53–0.55. Mặc dù mô hình có khả năng học được một số quy luật từ dữ liệu, nhưng hiệu năng vẫn còn hạn chế, đặc biệt trong việc nhận diện các phiên tăng giá, thể hiện qua confusion matrix với số lượng False Negative lớn. Ngoài ra, kết quả backtesting cho thấy chiến lược giao dịch dựa trên mô hình không vượt trội so với phương pháp mua và nắm giữ, thậm chí cho hiệu suất thấp hơn đáng kể. Điều này cho thấy bài toán dự đoán thị trường tài chính là rất phức tạp và đòi hỏi các mô hình cũng như đặc trưng mạnh hơn.

Tuy nhiên, điểm quan trọng nhất mà đề tài đạt được không chỉ nằm ở kết quả mô hình, mà còn ở quá trình triển khai. Thông qua bài nghiên cứu này, nhóm đã hiểu rõ và nắm vững cách sử dụng Apache Spark trong xử lý dữ liệu lớn, bao gồm các thao tác tiền xử lý dữ liệu, xây dựng pipeline với các thành phần như StringIndexer, VectorAssembler, cũng như huấn luyện và đánh giá mô hình học máy trên môi trường phân tán. Bên cạnh đó, nhóm cũng đã làm quen với cách tổ chức quy trình thực nghiệm, đánh giá mô hình theo thời gian và áp dụng vào bài toán thực tế.

Trong tương lai, để cải thiện hiệu quả mô hình, nhóm có thể mở rộng theo các hướng như bổ sung thêm đặc trưng (dữ liệu vĩ mô, tin tức, sentiment), thử nghiệm các mô hình nâng cao hơn như Gradient Boosting hoặc Deep Learning, cũng như tối ưu chiến lược giao dịch và quản lý rủi ro. Đây sẽ là những hướng phát triển tiềm năng nhằm nâng cao chất lượng dự đoán và khả năng ứng dụng thực tế của hệ thống.

TÀI LIỆU

- [1] J. Y. Zhu, B. Tang, and V. O. Li, “A five-layer architecture for big data processing and analytics,” *International Journal of Big Data Intelligence*, vol. 6, no. 1, pp. 38–49, 2019.
- [2] “Big data 1: What is big data? | ACCA Global.” [Online]. Available: <https://www.accaglobal.com/gb/en/student/exam-support-resources/fundamentals-exams-study-resources/f5/technical-articles/what-is-big-data.html>
- [3] “What are the 5 V’s of Big Data? | Definition TechTarget.” [Online]. Available: <https://www.techtarget.com/searchdatamanagement/definition/5-Vs-of-big-data>
- [4] A. Gandomi and M. Haider, “Beyond the hype: Big data concepts, methods, and analytics,” *International journal of information management*, vol. 35, no. 2, pp. 137–144, 2015.
- [5] M. Chen, S. Mao, and Y. Liu, “Big data: A survey,” *Mobile networks and applications*, vol. 19, no. 2, pp. 171–209, 2014.
- [6] F. Zhao, “Big data applications and mining in the healthcare field,” *J. Comput. Electr. Inf. Manag*, vol. 12, no. 1, pp. 27–31, 2024.
- [7] W. Raghupathi and V. Raghupathi, “Big data analytics in healthcare: promise and potential,” *Health information science and systems*, vol. 2, no. 1, p. 3, 2014.
- [8] S. Dash, S. K. Shakyawar, M. Sharma, and S. Kaushik, “Big data in healthcare: management, analysis and future prospects,” *Journal of big data*, vol. 6, no. 1, p. 54, 2019.
- [9] H. Nobanee, M. N. Dilshad, M. Al Dhanhani, M. Al Neyadi, S. Al Qubaisi, and S. Al Shamsi, “Big data applications the banking sector: a bibliometric analysis approach,” *Sage Open*, vol. 11, no. 4, p. 21582440211067234, 2021.
- [10] K. Rahul, R. K. Banyal, and N. Arora, “A systematic review on big data applications and scope for industrial processing and healthcare sectors,” *Journal of Big Data*, vol. 10, no. 1, p. 133, 2023.
- [11] S. A. Osinga, D. Paudel, S. A. Mouzakis, and I. N. Athanasiadis, “Big data in agriculture: Between opportunity and solution,” *Agricultural systems*, vol. 195, p. 103298, 2022.
- [12] A. Badshah, A. Daud, R. Alharbey, A. Banjar, A. Bukhari, and B. Alshemaimri, “Big data applications: overview, challenges and future,” *Artificial Intelligence Review*, vol. 57, no. 11, p. 290, 2024.

- [13] K. V. Shvachko, “Apache hadoop,” *The Scalability Update*, vol. 36, no. 3, pp. 7–13, 2011.
- [14] S. Salloum, R. Dautov, X. Chen, P. X. Peng, and J. Z. Huang, “Big data analytics on apache spark,” *International Journal of Data Science and Analytics*, vol. 1, no. 3, pp. 145–164, 2016.
- [15] C. Strauch, U.-L. S. Sites, and W. Kriha, “Nosql databases,” *Lecture Notes, Stuttgart Media University*, vol. 20, no. 24, p. 79, 2011.
- [16] N. Garg, *Apache kafka*. Packt Publishing, 2013.
- [17] A. Nambiar and D. Mundra, “An overview of data warehouse and data lake in modern enterprise data management,” *Big data and cognitive computing*, vol. 6, no. 4, p. 132, 2022.
- [18] M. Zaharia, R. S. Xin, P. Wendell, T. Das, M. Armbrust, A. Dave, X. Meng, J. Rosen, S. Venkataraman, M. J. Franklin *et al.*, “Apache spark: a unified engine for big data processing,” *Communications of the ACM*, vol. 59, no. 11, pp. 56–65, 2016.
- [19] H. Karau, A. Konwinski, P. Wendell, and M. Zaharia, *Learning spark: lightning-fast big data analysis*. "O'Reilly Media, Inc.", 2015.
- [20] “Apache Spark Architecture Explained - Spark Playground.” [Online]. Available: <https://www.sparkplayground.com/tutorials/spark-theory/spark-architecture>
- [21] “9000+ Tickers of Stock Market Data (Full History).” [Online]. Available: <https://www.kaggle.com/datasets/jakewright/9000-tickers-of-stock-market-data-full-history>